PI_Curs_07

7.0

Absolut! Lecția **7. Segmentarea Semantică (Semantic Segmentation)** reprezintă următoarea etapă majoră în înțelegerea imaginilor, trecând dincolo de simpla încadrare a obiectelor cu cutii dreptunghiulare.

Ghid de Studiu Complet: 7. Segmentarea Semantică (Semantic Segmentation)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Capitolul 7 introduce **Segmentarea Semantică** ca o problemă crucială în viziunea computerizată. Dacă sarcina de detecție a obiectelor (Capitolul 6) folosește **cutii de delimitare dreptunghiulare** pentru a eticheta și prezice obiecte, Segmentarea Semantică ridică nivelul de înțelegere, recunoscând și înțelegând conținutul imaginilor la **nivel de pixel**. Obiectivul este de a împărți o imagine în regiuni aparținând diferitelor clase semantice. Granițele regiunilor etichetate și prezise sunt **mult mai detaliate (*more fine-grained*)** comparativ cu detecția obiectelor. Acest capitol introduce, de asemenea, două sarcini conexe și importante: Segmentarea Instanțelor și Segmentarea Panoptică.

Care sunt cele 3 idei principale pe care trebuie neapărat să le rețin?

1. **Diferența Esențială:** Segmentarea Semantică lucrează la **nivel de pixel**, clasificând fiecare pixel cu o etichetă de clasă semantică.
2. **Precizie:** Rezultatele (granițele) sunt **mai detaliate** decât în cazul cutiilor de delimitare.
3. **Sarcini Conexe:** Există două sarcini similare Segmentării Semantice: **Segmentarea Instanțelor** (distinge instanțele aceluiași obiect) și **Segmentarea Panoptică** (combină Segmentarea Semantică și Segmentarea Instanțelor).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

7. Segmentarea Semantică

Definirea Segmentării Semantice

- Atunci când se discută despre detecția obiectelor în Capitolul 6, se utilizează cutii de delimitare dreptunghiulare pentru a eticheta și prezice obiecte în imagini.

- Segmentarea Semantică se concentrează pe problema împărțirii unei imagini în regiuni aparținând diferitelor **clase semantice**.
- Spre deosebire de detecția obiectelor, segmentarea semantică recunoaște și înțelege conținutul imaginilor la **nivel de pixel**.
- Etichetarea și predicția regiunilor semantice se fac la nivel de pixel.
- Granițele regiunilor la nivel de pixel etichetate în segmentarea semantică sunt **evident mai detaliate (*more fine-grained*)** în comparație cu detecția obiectelor.

Exemplu Vizual (Figura 3 și 4):

- O imagine este împărțită în regiuni etichetate: Cer (*Sky*), Copaci (*Trees*), Vacă (*Cow*), Iarbă (*Grass*).

Sarcini Importante Similare Segmentării Semantice

Există două sarcini importante în viziunea computerizată similare Segmentării Semantice:

1. **Segmentarea Instanțelor (*Instance segmentation*):**
   - Este numită și **detecție și segmentare simultană (*simultaneous detection and segmentation*)**.
   - Obiectivul este de a recunoaște regiunile la nivel de pixel ale **fiecărei instanțe de obiect** dintr-o imagine.
   - Diferit de segmentarea semantică, segmentarea instanțelor trebuie să distingă nu numai semantica, ci și **diferitele instanțe de obiect**.
   - Exemplu: Dacă sunt două vaci în imagine, Segmentarea Instanțelor trebuie să distingă cărui pixel aparține fiecare dintre cele două vaci.
2. **Segmentarea Panoptică (*Panoptic segmentation*):**
   - Această metodă oferă etichete pentru obiecte **nenumărabile (*uncountable*)** numite "stuff" (ex: cer, iarbă) – similar segmentării semantice.
   - De asemenea, diferențiază pixelii proveniți din **instanțe diferite** ale aceleiași clase (obiecte numărabile numite "things," ex: vaci) – similar segmentării instanțelor.
   - Exemplu: Imaginea este etichetată cu "Cow #1", "Cow #2" (instanțe) și "Sky", "Trees", "Grass" (stuff).

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Segmentare la Nivel de Pixel (Pixel-Level)

- **Analogie (Harta Detaliată vs. Harta Generală):** Detecția obiectelor (bounding box) este ca și cum ai desena o hartă generală a unei țări, unde doar încadrezi orașele.

Segmentarea Semantică este ca și cum ai desena o hartă topografică foarte detaliată, unde fiecare părticică de teren (pixel) este etichetată cu exactitate (pădure, lac, câmp).
- *Asta se întâmplă deoarece:* Segmentarea Semantică necesită un nivel de precizie mult mai mare. În loc să prezică 4 coordonate (bounding box), modelul trebuie să prezică o etichetă (clasa) pentru **fiecare pixel** din imagine.

Concept Dificil: Stuff vs. Things (Segmentarea Panoptică)

- **Clarificare:** Această distincție este cheia Segmentării Panoptice:
  - **Things (Obiecte numărabile):** Obiecte discrete care pot fi numărate și delimitate clar (ex: oameni, mașini, animale). Pentru acestea, trebuie să distingem instanțele (omul 1 vs. omul 2).
  - **Stuff (Obiecte nenumărabile):** Regiuni amorfe, care ocupă adesea un fundal mare (ex: cerul, iarba, drumul). Pentru acestea, nu contează "instanțele"; contează doar clasa semantică generală.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Precizie vs. Efort de Etichetare):**
  - **CAUZĂ:** Dorința de a înțelege o scenă cu detalii mult mai fine decât oferă un simplu dreptunghi.
  - **EFECT:** Dezvoltarea Segmentării Semantice, care, la rândul ei, necesită **seturi de date mult mai laborioase** (etichetarea manuală a fiecărui pixel este mult mai costisitoare decât desenarea câtorva cutii de delimitare).
- **Conexiunea cu Capitolul 6 (Object Detection):** Sarcinile de Segmentare Instanțe și Segmentare Panoptică sunt, de fapt, o **combinație** între detecția obiectelor și segmentarea semantică. Ele păstrează capacitatea de a identifica instanțe individuale (ca în detecție) dar o fac cu precizia la nivel de pixel (ca în segmentare).

---

5. EXERCITII (Teorie și Diferențiere)

Deoarece sursa se axează pe definirea conceptelor și diferențele dintre sarcini, exercițiile vor fi axate pe analiza comparativă.

Exercițiul D.1: Diferențiere Fundamentală (Tabular)

Completați următorul tabel comparativ, bazându-vă pe informațiile despre Segmentarea Semantică și Detecția Obiectelor:

Caracteristică	Detecția Obiectelor (Capitolul 6)	Segmentarea Semantică (Capitolul 7)
Ieșirea Predicției	Casetă de delimitare (Bounding Box)	Etichetă de clasă pentru **fiecare pixel**
Granularitate	Cutii dreptunghiulare	Granițe la nivel de pixel, **mai detaliate**
Recunoaștere Instanțe?	Nu distinge instanțele (face parte din clasă)	Nu distinge în mod implicit instanțele

Rezolvare D.1:

Caracteristică	Detecția Obiectelor (Capitolul 6)	Segmentarea Semantică (Capitolul 7)
Ieșirea Predicției	Casetă de delimitare (Bounding Box)	Etichetă de clasă pentru **fiecare pixel**
Granularitate	Cutii dreptunghiulare	Granițe la nivel de pixel, **mai detaliate**
Recunoaștere Instanțe?	Nu distinge instanțele (face parte din clasă)	Nu distinge în mod implicit instanțele

Exercițiul D.2: Identificarea Sarcinii (Conceptual)

O imagine conține două vaci.

1. Ce tip de segmentare trebuie utilizat pentru a eticheta toți pixelii care aparțin clasei "Vacilor" în general, fără a diferenția Vacă #1 de Vacă #2, și restul pixelilor ca "Iarbă" sau "Cer"?
2. Ce tip de segmentare trebuie utilizat pentru a distinge pixelii care aparțin Vacii #1 de pixelii care aparțin Vacii #2?

Rezolvare D.2:

1. **Segmentarea Semantică:** Obiectivul său este de a recunoaște și eticheta regiunile aparținând diferitelor clase semantice. Dacă sunt două vaci, ambele ar primi aceeași etichetă "Vacă" (sau *Cow*), fără a fi distinse ca instanțe separate.
2. **Segmentarea Instanțelor (*Instance segmentation*):** Această sarcină este diferită de segmentarea semantică, deoarece necesită distingerea diferitelor **instanțe de obiect** (Vacă #1 vs. Vacă #2) la nivel de pixel.

Exercițiul D.3: Clasificarea Obiectelor (Panoptic Segmentation)

În contextul Segmentării Panoptice, clasificați următoarele elemente în categoriile "Things" (obiecte numărabile) și "Stuff" (obiecte nenumărabile):

1. Cer (*Sky*)
2. Vacă (*Cow*)
3. Copaci (*Trees*)

Rezolvare D.3:

1. **Cer (*Sky*):** "Stuff" (Obiect nenumărabil).
2. **Vacă (*Cow*):** "Things" (Obiect numărabil, necesită diferențierea instanțelor).
3. **Copaci (*Trees*):** "Stuff" (Obiect nenumărabil).

7.1

Lecția **7.1 Upsampling** (Eșantionarea în Sus) este o componentă arhitecturală esențială pentru realizarea Segmentării Semantice. Deoarece rețelele neuronale convoluționale (CNN) reduc de obicei dimensiunea spațială a datelor (prin *downsampling*), această tehnică devine necesară pentru a readuce harta de *feature*-uri la rezoluția inițială a imaginii, permițând predicția la nivel de pixel.

Ghid de Studiu Complet: 7.1 Upsampling

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 7.1 abordează necesitatea de a crește **dimensiunile spațiale (înălțimea și lățimea)** ale hărților de *feature*-uri (*intermediate feature maps*) în rețelele neuronale convoluționale (CNN). Straturile CNN obișnuite (convoluționale și *pooling*) reduc în mod tipic (*downsample*) dimensiunile spațiale sau le mențin neschimbate. Totuși, în Segmentarea Semantică, care clasifică la **nivel de pixel**, este vital ca dimensiunile spațiale ale intrării și ieșirii să fie identice, astfel încât dimensiunea canalului la un pixel de ieșire să conțină rezultatul clasificării pentru pixelul de intrare din aceeași poziție spațială. Pentru a realiza acest lucru, se folosesc tipuri noi de straturi CNN care pot **crește (*upsample*)** dimensiunile spațiale.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Necesitatea Upsampling-ului:** CNN-urile obișnuite reduc dimensiunile (downsampling), dar Segmentarea Semantică necesită ca dimensiunile spațiale ale intrării și ieșirii să fie egale.
2. **Scopul de Bază:** Un pixel de ieșire trebuie să conțină rezultatul clasificării pentru pixelul de intrare corespunzător la aceeași poziție spațială.
3. **Metode Simple de Up-pooling:** *Unpooling* (ex. *bed of nails*, *nearest neighbor*) și *Max-unpooling* (bazat pe poziția maximului).
4. **Metode Avansate: Interpolarea** (biliniară, bicubică) și **Convoluția Transpusă (*Transposed convolution* sau *Fractionally-strided convolution*)**, care inversează operația de *downsampling*.

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

7.1 Upsampling

Necesitatea Upsampling-ului

- Straturile CNN văzute anterior (convoluționale, Secțiunea 3.2, și *pooling*, Secțiunea 3.5) reduc (*downsample*) sau mențin neschimbate dimensiunile spațiale ale intrării.
- În segmentarea semantică, care clasifică la nivel de pixel, este necesar ca dimensiunile spațiale ale intrării și ieșirii să fie aceleași.
- Dimensiunea canalului la un pixel de ieșire poate reține rezultatele clasificării pentru pixelul de intrare de la aceeași poziție spațială.
- Pentru a obține acest lucru, se utilizează straturi CNN care pot **crește (*upsample*)** dimensiunile spațiale ale hărților de *feature*-uri intermediare.

Metode de Unpooling (Reconstrucție Simplă)

1. **Unpooling – bed of nails:**
   - Elementele din tensorul de intrare (ex. $C \times 2 \times 2$) sunt plasate în pozițiile corespunzătoare din tensorul de ieșire (ex. $C \times 4 \times 4$).
   - Celelalte poziții sunt umplute cu **zero**. (Exemplu: 1 se plasează în colțul stânga sus al blocului său 2×2, restul sunt 0).
2. **Unpooling – nearest neighbor:**
   - Valoarea unui element de intrare este **copiată** în toți pixelii învecinați din regiunea corespunzătoare de ieșire. (Exemplu: Elementul 1 din intrarea 2×2 este copiat de patru ori în regiunea sa 2×2 din ieșirea 4×4).
3. **Max-unpooling:**
   - Se utilizează pentru a reface o hartă de *feature*-uri după *max-pooling*.
   - Se memorează poziția care a dat valoarea maximă în stratul de *max-pooling*.
   - La *max-unpooling*, valorile sunt plasate **doar în pozițiile memorate**.
   - Fiecare strat de *downsampling* este asociat cu un strat de *upsampling*.

Metode de Interpolare

1. **Interpolare Biliniară (*Bilinear interpolation*):**
   - Folosește cei **patru** cei mai apropiați vecini ai unui pixel (x, y) pentru a construi aproximări liniare.
2. **Interpolare Bicubică (*Bicubic interpolation*):**
   - Folosește cei **șaisprezece** cei mai apropiați vecini ai unui pixel (x, y) pentru a construi aproximări cubice.

Convoluția Transpusă (*Transposed convolution*)

- Numită și **Fractionally-strided convolution**, este introdusă pentru a inversa operațiile de *downsampling* prin convoluție.
- **Operația de Bază (Stride 1, Fără Padding):**
  - Se dă un tensor de intrare $n_h \times n_w$ și un kernel $k_h \times k_w$.
  - Kernelul este glisat de n_w ori pe rând și n_h ori pe coloană.
  - Se obține un total de $n_h n_w$ rezultate intermediare.
  - Fiecare rezultat intermediar este un tensor $(n_h + k_h - 1) \times (n_w + k_w - 1)$ inițializat cu zero.
- **Calculul:** Fiecare element din tensorul de intrare este înmulțit cu kernelul, iar tensorul rezultat $k_h \times k_w$ înlocuiește o porțiune din tensorul intermediar.
- **Ieșirea Finală:** Toate rezultatele intermediare sunt însumate pentru a produce ieșirea finală.
- **Diferențe față de Convoluția Standard:**
  - Convoluția standard reduce elementele de intrare; convoluția transpusă **difuzează (*broadcasts*)** elementele de intrare via kernel, producând o ieșire **mai mare** decât intrarea.
  - *Padding*-ul este aplicat la **ieșire**, nu la intrare.
  - *Stride*-ul este specificat pentru rezultatele intermediare (**adică, ieșire**), nu pentru intrare.
- **Efectul Stride-ului:** Dacă se schimbă *stride*-ul de la 1 la $s = 2$ (folosind aceiași tensori de intrare și kernel), se măresc atât înălțimea, cât și lățimea tensorilor intermediari, mărind astfel tensorul de ieșire.
- **Mărirea Generală a Dimensiunilor:** În general, pentru un *stride* s, *padding* $s/2$ (asumând $s/2$ integer), și un kernel $2s$, convoluția transpusă va mări înălțimea și lățimea intrării cu un **factor de** s.
- **Canale Multiple:** Convoluția transpusă funcționează la fel ca și convoluția standard pentru canale multiple de intrare și ieșire.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Convoluția Transpusă (*Transposed Convolution*)

- **Explicație Simplificată (Desenul Invers):** O convoluție normală (ca un *downsampling*) comprimă o zonă mare într-un singur pixel. Convoluția transpusă face invers: ia valoarea unui singur pixel și o *descompune* într-o zonă mai mare de pixeli de ieșire. Acesta nu este un proces de inversare matematică perfectă (nu neapărat o deconvoluție), ci mai degrabă un mod de a realiza *upsampling* **învățabil**.
- *Asta se întâmplă deoarece:* Interpolarea (biliniară/bicubică) folosește reguli matematice fixe pentru a umple spațiul dintre pixeli. Convoluția transpusă, în schimb, folosește

greutățile învățate (kernel-ul) pentru a distribui valoarea de intrare în zona de ieșire. Acest lucru oferă rețelei flexibilitatea de a învăța cea mai bună modalitate de a reconstrui detaliile spațiale pierdute.

Concept Dificil: Max-Unpooling

- **Analogie (Găsirea Locului Pierdut):** Imaginați-vă că ați redus o imagine folosind *max pooling*: ați luat cel mai strălucitor punct dintr-o zonă 2×2 și l-ați păstrat. La *max-unpooling*, noi nu ghicim unde să punem acea valoare înapoi; folosim o "hartă de memorie" stocată în timpul *max pooling* pentru a plasa valoarea exact în poziția inițială. Acest lucru este crucial pentru reconstruirea precisă a granițelor obiectelor, deoarece cele mai active *feature*-uri sunt restaurate în locația lor originală.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Rezoluție și Sarcina):**
  - **CAUZĂ:** Sarcina Segmentării Semantice necesită clasificare la nivel de pixel.
  - **EFECT:** Este necesar ca rezoluția finală să corespundă rezoluției inițiale. Deoarece arhitecturile CNN reduc rezoluția, este necesară introducerea blocurilor de *upsampling* (ca un *decoder*) pentru a compensa pierderea spațială indusă de blocurile de *downsampling* (ca un *encoder*).
- **Conexiunea cu Secțiunea 3.5 (Pooling):** *Max-unpooling* (Secțiunea 7.1) este direct corelat cu *Max-pooling* (Secțiunea 3.5). Cele două straturi trebuie să fie asociate pentru a putea recupera informația despre poziția elementului maxim.

---

5. EXERCITII (Calcul și Analiză Dimensională)

Exercițiul U.1: Analiza Max-Unpooling

Se consideră o hartă de *feature*-uri de intrare 4×4 care este supusă unui *Max-Pooling* de 2×2 cu *stride* 2. Rezultatul *Max-Pooling*-ului este 2×2 și conține valorile [5, 6; 7, 8]. (Conform Fig. 11, Sursa)

1. Dacă se efectuează operația de *Max-Unpooling* pe harta de *feature*-uri 2×2 rezultată, care va fi valoarea maximă plasată în regiunea corespunzătoare elementului 8 (dreapta jos) și care este poziția (relativă, de la 0 la 3) în care va fi plasată această valoare?
2. Ce se întâmplă cu celelalte poziții nefolosite în timpul *Max-Unpooling*-ului (cele care nu au dat maximul)?

Rezolvare U.1:

1. **Valoarea și Poziția:** Valoarea plasată este **8**. Poziția în care se plasează 8 este poziția care a dat maximul în blocul corespunzător de *downsampling* (dreapta jos). În exemplul 4×4 din sursă, această valoare (8) este plasată în colțul din dreapta jos al regiunii 2×2 corespunzătoare.
2. **Pozițiile nefolosite:** Pozițiile din harta de *feature*-uri de ieșire care nu corespund poziției maxime memorate în timpul *Max-Pooling*-ului sunt **umplute cu zero**.

Exercițiul U.2: Analiza Dimensională (Convoluția Transpusă)

Se aplică o convoluție transpusă pe un tensor de intrare 4×4. Se utilizează un *stride* $s = 2$, *padding* 1, și un kernel de 2×2 (ignorând factorul $2s$).

1. Conform regulii generale (pentru s, $s/2$ padding, și kernel $2s$), cu ce factor ar trebui să crească înălțimea și lățimea tensorului de intrare?
2. Dacă se folosește un strat convoluțional standard f care transformă $X \rightarrow$, și apoi se creează un strat convoluțional transpus g cu aceiași *hyperparametri* (cu excepția numărului de canale de ieșire, care este egal cu numărul de canale din X), care va fi forma lui $g()$ comparativ cu X?

Rezolvare U.2:

1. **Factorul de Creștere:** Pentru un *stride* s, convoluția transpusă va crește înălțimea și lățimea intrării cu un **factor de** s. În acest caz, factorul de creștere este $\mathbf{2}$. (Ieșirea ar fi 8×8).
2. **Forma lui $g()$:** Dacă se aplică $X \rightarrow$ folosind convoluția f și se creează convoluția transpusă g cu aceiași *hyperparametri* (cu excepția numărului de canale de ieșire egal cu numărul de canale din X), atunci $g()$ va avea **aceeași formă spațială ca** X.

Exercițiul U.3: Diferențiere Conceptuală (Interpolare vs. Unpooling)

Comparați metodele de *unpooling – bed of nails* și *unpooling – nearest neighbor* din punct de vedere al informațiilor plasate în pixeli de ieșire, așa cum sunt ilustrate în Figurile 3 și 5.

Rezolvare U.3:

1. **Unpooling – bed of nails:**
   - Plasează valorile de intrare **doar în poziția din stânga sus** a regiunii de ieșire corespunzătoare 2×2.
   - Umple restul pixelilor învecinați cu **zero**.
2. **Unpooling – nearest neighbor:**
   - **Copiază** valoarea elementului de intrare în **toți** pixelii învecinați din regiunea de ieșire corespunzătoare 2×2.
   - Această metodă oferă o hartă de *feature*-uri mai densă (fără zerouri), dar introduce o artefacte vizibile (*blockiness*).

7.2

Suntem la lecția **7.2 Fully convolutional networks (Rețele Complet Convoluționale - FCN)**, care descrie arhitectura fundamentală ce a permis segmentarea semantică eficientă la nivel de pixel, bazându-se pe tehnica de *upsampling* discutată anterior.

Ghid de Studiu Complet: 7.2 Fully convolutional networks (FCN)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 7.2 se concentrează pe **Rețelele Complet Convoluționale (FCN)**, modelul standard utilizat pentru **Segmentarea Semantică**. Spre deosebire de rețelele neuronale convoluționale (CNN) folosite anterior pentru clasificare sau detecție, un FCN transformă înălțimea și lățimea hărților de *feature*-uri intermediare **înapoi** la dimensiunile imaginii de intrare. Acest lucru este realizat prin utilizarea stratului de **convoluție transpusă** (introdus în Secțiunea 7.1). Obiectivul final este o **corespondență unu-la-unu la nivel de pixel** între ieșirea clasificării și imaginea de intrare, unde dimensiunea canalului la orice pixel de ieșire conține rezultatele clasificării pentru pixelul de intrare din aceeași poziție spațială. FCN-urile pot fi obținute prin **transformarea straturilor complet conectate** (*fully connected layers*) în straturi convoluționale.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Scopul FCN:** Transformă înălțimea și lățimea hărților de *feature*-uri înapoi la dimensiunile imaginii de intrare, realizând clasificarea la **nivel de pixel**.
2. **Mecanismul de Upsampling:** Această transformare dimensională este realizată prin stratul de **convoluție transpusă**.
3. **Conversia (Convolutionalization):** Straturile complet conectate pot fi văzute ca și convoluții cu *kernel*-uri care acoperă întreaga regiune de intrare, permițând transformarea rețelelor de clasificare în FCN-uri.
4. **Antrenamentul Ierarhic (Fuzionarea Feature-urilor):** FCN-urile folosesc antrenamentul ierarhic pentru a combina informațiile de nivel înalt (*coarse*, semantica generală) cu detaliile de nivel mijlociu și scăzut (*fine*, granițele).

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

7.2 Rețele Complet Convoluționale (FCN)

Designul de Bază și Principiul FCN

- Segmentarea semantică clasifică imaginile la nivel de pixel.

- Un FCN folosește o rețea neuronală convoluțională pentru a transforma pixelii imaginii în clase de pixeli.
- Spre deosebire de CNN-urile anterioare (pentru clasificare sau detecție), un FCN transformă înălțimea și lățimea hărților de *feature*-uri intermediare înapoi la cele ale imaginii de intrare.
- Aceasta este realizată prin stratul de **convoluție transpusă**.
- Rezultatul este o **corespondență unu-la-unu la nivel de pixel**: dimensiunea canalului la orice pixel de ieșire deține rezultatele clasificării pentru pixelul de intrare aflat la aceeași poziție spațială.

Arhitectura și Procesul (Figura 3)

Designul de bază al modelului FCN implică:

1. Utilizarea unui CNN pentru a extrage *feature*-uri din imaginea de intrare.
2. Transformarea numărului de canale în numărul de clase prin intermediul unui **strat convoluțional de** 1×1.
3. Transformarea finală a înălțimii și lățimii hărților de *feature*-uri la cele ale imaginii de intrare, realizată de convoluția transpusă.
4. Stratul de *upsampling* (convoluția transpusă) este inițializat cu **interpolare biliniară**.
5. Ieșirea modelului are aceeași înălțime și lățime ca imaginea de intrare, iar canalul de ieșire conține clasele prezise pentru pixelul de intrare la aceeași poziție spațială.

Conversia Stratului Complet Conectat (Convolutionalization)

- Straturile complet conectate pot fi văzute ca **convoluții cu *kernel*-uri care acoperă întreaga regiune de intrare**.
- Această transformare face ca rețelele de clasificare să devină rețele complet convoluționale (FCN) care acceptă intrări de **orice dimensiune** și oferă hărți de clasificare ca ieșire.
- Această transformare este ilustrată în Figura 4.

Antrenamentul Ierarhic și Fuzionarea Feature-urilor

- Se folosește un **antrenament ierarhic** în care rețeaua este inițial antrenată pe *feature*-uri de **nivel înalt** și apoi *fine-tuned* pe *feature*-uri de nivel mijlociu și scăzut.
- Importanță: Permite rețelei să învețe detaliile de nivel mijlociu și scăzut (granițe fine) ale imaginii, pe lângă informațiile de nivel înalt (semantica generală).
- **Pașii Ierarhici (FCN-32s, FCN-16s, FCN-8s):**
  - Inițial, se prezice masca de segmentare din *feature*-urile de nivel înalt (**FCN-32s**).
  - Apoi, se prezice masca din *feature*-urile de nivel mijlociu (**FCN-16s**).
  - În final, se prezice masca din *feature*-urile de nivel scăzut (**FCN-8s**).
- **Mecanismul de Ieșire (Figura 5):**

    - **FCN-32s (Linie continuă):** Rețea cu un singur flux care realizează *upsample* de 32 de ori pentru a prezice pixeli într-un singur pas.
    - **FCN-16s (Linie întreruptă):** Combină predicțiile de la ultimul strat (*pool5*) și stratul *pool4*, utilizând *stride* 16, reținând detaliile mai fine.
    - **FCN-8s (Linie punctată):** Adaugă predicțiile de la *pool3*, utilizând *stride* 8, oferind o precizie și mai mare.
- Fuzionarea informațiilor din straturi cu *stride*-uri diferite îmbunătățește detaliul segmentării.

Antrenarea FCN-urilor

- Funcția de pierdere (*loss function*) și calculul acurateții nu sunt esențial diferite de cele din clasificarea imaginilor.
- Se utilizează operația **softmax** pentru a prezice clasa.
- Se folosesc 21 de canale de ieșire pentru a prezice clasa, pe baza setului de date **Pascal VOC2012** (20 de clase + clasa *background*).
- **Funcția de pierdere** este **entropia încrucișată (*cross-entropy*) per pixel**.
- Acuratețea este calculată pe baza corectitudinii clasei prezise **pentru toți pixelii**.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: FCN (Fully Convolutional Network)

- **Explicație Simplificată (Rețeaua cu Formă de U):** Un FCN poate fi vizualizat ca un "U". Partea descendentă (*encoder*) este CNN-ul standard care extrage *feature*-uri și comprimă spațial imaginea (downsampling). Partea ascendentă (*decoder*) folosește **convoluția transpusă** pentru a de-comprima spațial *feature*-urile, readucând rezoluția la cea inițială.

Concept Dificil: Fuzionarea Ierarhică a Feature-urilor (FCN-32s, FCN-16s, FCN-8s)

- **Analogie (Echipa de Detalii):** Imaginați-vă că aveți o hartă a unei clădiri:
    - **FCN-32s (Nivel Înalt):** Vede doar forma generală a clădirii (este o casă, nu o fabrică). Este rapidă, dar inexactă la colțuri.
    - **FCN-16s (Nivel Mediu):** Combină informația generală cu *feature*-uri mai detaliate (de la *pool4*), ajutând la trasarea pereților.
    - **FCN-8s (Nivel Scăzut):** Combină cu *feature*-uri de la *pool3* (cele mai fine detalii), permițând modelului să traseze cu exactitate marginile ferestrelor și ușilor.
    - *Asta se întâmplă deoarece:* Straturile superioare (pool5) conțin semantică robustă, dar au rezoluție spațială scăzută. Straturile inferioare (pool3) au rezoluție înaltă, dar

conțin *feature*-uri primare (margini). Fuzionarea combină **"Ce este obiectul"** (din *pool5*) cu **"Unde sunt granițele exacte"** (din *pool3*).

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Conversia Straturilor FC):**
  - **CAUZĂ:** Transformarea straturilor complet conectate în convoluții.
  - **EFECT:** Rețeaua FCN poate accepta imagini de **orice dimensiune** (nu doar o dimensiune fixă), generând o hartă de clasificare la ieșire. Aceasta este o îmbunătățire semnificativă față de CNN-urile tradiționale de clasificare care necesitau o dimensiune de intrare fixă din cauza stratului FC.
- **Conexiunea cu Secțiunea 7.1:** Utilizarea convoluției transpusă, inițializată cu interpolare biliniară, este o aplicație directă a tehnicilor de *upsampling* pentru a realiza reconstrucția dimensională necesară.

---

5. EXERCITII (Calcul și Analiză Dimensională)

Exercițiul C.1: Calculul Canalelor de Ieșire

Se dorește antrenarea unui FCN pe un set de date similar cu Pascal VOC2012, dar care include doar 10 clase de obiecte.

1. Câte canale de ieșire (output channels) trebuie să aibă stratul final de convoluție transpusă pentru a prezice clasa fiecărui pixel?
2. Dacă un pixel de ieșire (de exemplu, la poziția 50, 50) returnează un vector de probabilități prin *softmax* pe aceste canale, ce reprezintă cea mai mare valoare din acest vector?

Rezolvare C.1:

1. **Numărul de Canale de Ieșire:** Numărul de canale trebuie să fie egal cu numărul de clase de obiecte plus clasa de *background*.

$$\text{Canale Ieșire} = 10 \text{ (clase obiecte)} + 1 \text{ (bacground)} = \mathbf{11}$$

2. **Reprezentanța Valorii Maxima:** Cea mai mare valoare din vectorul de probabilități returnat de *softmax* reprezintă **clasa prezisă** (eticheta semantică) pentru pixelul de intrare situat la aceeași poziție spațială **(50, 50)**.

Exercițiul C.2: Analiza Stride-ului în Antrenamentul Ierarhic

În arhitectura FCN (Figura 5), se folosesc trei variante: FCN-32s, FCN-16s și FCN-8s. Explicați ce indică valoarea numerică (32, 16, 8) în contextul straturilor CNN și cum

contribuie FCN-8s la îmbunătățirea rezultatului final.

Rezolvare C.2:

1. **Semnificația Stride-ului (32, 16, 8):** Valoarea numerică indică **stride-ul de *upsampling*** necesar pentru a readuce harta de *feature*-uri la rezoluția inițială a imaginii. Stride-ul este determinat de gradul total de *downsampling* realizat de straturile de *pooling* până la acel punct (pool5 reduce rezoluția de 32 de ori, pool4 de 16 ori, pool3 de 8 ori).
2. **Contribuția FCN-8s:** FCN-8s (linia punctată) folosește *feature*-uri de la stratul **pool3** (care are o rezoluție spațială mai înaltă, *low-level features*). Deoarece pool3 conține detalii spațiale mai fine, fuzionarea sa cu predicțiile de nivel superior permite rețelei să învețe și să prezică **detalii fine** și să traseze **granițe mai precise** pentru masca de segmentare.

7.3

Lecția **7.3 SegNet** prezintă o arhitectură specializată de rețea neuronală convoluțională, proiectată specific pentru sarcina de segmentare semantică. SegNet este un model eficient, bazat pe o structură simetrică de tip **Encoder-Decoder**, a cărui inovație principală constă în utilizarea inteligentă a **indicilor de *max-pooling*** pentru a realiza o reconstrucție precisă a detaliilor spațiale și a granițelor obiectelor.

Ghid de Studiu Complet: 7.3 SegNet

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

SegNet este o rețea de segmentare semantică bazată pe o arhitectură **Complet Convoluțională Encoder-Decoder**. Ideea de bază este de a realiza o **eșantionare în sus (*upsampling*) pas cu pas** pentru a transforma harta de *feature*-uri compactă înapoi la dimensiunea imaginii de intrare. Rețeaua Encoder este formată din primele **13 straturi convoluționale ale rețelei VGG-16**, din care straturile complet conectate sunt eliminate. Elementul definitoriu al SegNet, care asigură o **delineare vitală a granițelor**, este mecanismul prin care Decoder-ul folosește **indicii de *max-pooling* memorați** de Encoder pentru a realiza *Max-unpooling*.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Arhitectură de Bază:** SegNet folosește o structură simetrică **Encoder-Decoder** complet convoluțională, eliminând straturile complet conectate.
2. **Bază VGG-16:** Encoder-ul este format din **13 straturi convoluționale**, inițializate adesea cu ponderi de la VGG-16 antrenat pe ImageNet.
3. **Inovația Cheie (Max-unpooling):** Pentru a păstra detaliile spațiale (granițele), Encoder-ul **memorează indicii de *max-pooling***, iar Decoder-ul folosește acești indici pentru a realiza **Max-unpooling**.

4. **Procesul de Decodare:** Max-unpooling-ul inițial produce hărți de *feature*-uri **rare (*sparse*)**, care sunt apoi rafinate în hărți **dense** folosind straturi convoluționale antrenabile.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

7.3 SegNet – Arhitectura Generală

- Ideea SegNet este de a realiza **eșantionarea în sus pas cu pas**.
- Arhitectura constă dintr-o rețea **Encoder** și o rețea **Decoder** corespunzătoare, urmate de un strat final de clasificare la nivel de pixel.
- Arhitectura este complet convoluțională, **nu există straturi complet conectate**.
- Antrenarea poate fi inițializată prin transferul ponderilor antrenate pe seturi de date mari (e.g., ImageNet).

Rețeaua Encoder

- Encoder-ul este compus din **13 straturi convoluționale**.
- Acestea corespund primelor **13 straturi convoluționale din rețeaua VGG-16**.
- Straturile complet conectate sunt **eliminate** (discarded) pentru a reține hărți de *feature*-uri de rezoluție mai înaltă la cea mai profundă ieșire a encoder-ului.
- **Procesarea Datelor:** Fiecare encoder efectuează:
  - Convoluție, care produce un set de hărți de *feature*-uri.
  - **Normalizare pe Batch (*Batch normalization*)**.
  - Aplicarea element-wise a funcției de activare **ReLU**.
- **Downsampling și Pierderea Spațială:** Urmează operația de **max-pooling** cu o fereastră de 2×2 și un *stride* **2**, rezultând o sub-eșantionare a ieșirii cu un **factor de 2**.
- Deși *max-pooling*-ul oferă invarianță la translație (benefică clasificării), el cauzează o **pierdere de rezoluție spațială**.
- Această pierdere de detalii de graniță nu este benefică pentru segmentare, unde **delinearea graniței este vitală**.
- **Mecanismul de Stocare:** Pentru a stoca informațiile de graniță eficient înainte de sub-eșantionare, se stochează **doar indicii de *max-pooling*** (locațiile valorii maxime din fiecare fereastră de *pooling*) pentru fiecare hartă de *feature*-uri encoder.

Rețeaua Decoder

- Rețeaua Decoder are **13 straturi**, fiecare corespunzând unui strat encoder.
- **Operația Cheie (Max-unpooling):** Decoder-ul realizează *upsampling* pe hărțile sale de *feature*-uri de intrare utilizând **indicii de *max-pooling* memorați** din straturile encoder corespunzătoare. Aceasta este operația de **max-unpooling**.

- **Rezultat Intermediar:** Acest pas produce hărți de *feature*-uri **rare (*sparse feature maps*)**.
- **Rafinarea:** Hărțile rare sunt procesate folosind **straturi convoluționale antrenabile** (trainable convolutional layers) pentru a produce hărți de *feature*-uri **dense**. Scopul straturilor convoluționale este de a **rafina *upsampling*-ul**.
- După rafinare se aplică un pas de **normalizare pe batch**.
- **Excepție la Primul Decoder:** Decoder-ul care corespunde primului encoder (cel mai apropiat de imaginea de intrare) produce o hartă de *feature*-uri multi-canal cu **3 canale** (deoarece intrarea este imaginea RGB), spre deosebire de ceilalți decodere.

Clasificarea Finală

- Ieșirea hărții de *feature*-uri de înaltă dimensiune de la decoderele finale este transmisă unui **clasificator *softmax* antrenabil**.
- Acest clasificator produce **probabilități de clasă pentru fiecare pixel în mod independent** (pixelwise classification).

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Indicii de Max-Pooling Memorati (Pooling Indices)

- **Explicație Simplificată (Harta de Reconstrucție):** În timpul *max-pooling*-ului (când se comprimă o regiune 2×2 într-un singur pixel), se pierde informația spațială. Indicii de *max-pooling* sunt ca o notă de subsol: ei spun exact **unde** în regiunea 2×2 originală se afla valoarea maximă care a fost păstrată.
- *Asta se întâmplă deoarece:* Această informație stocată (indicii) este cheia Segmentării Semantice. Când Decoder-ul face *Max-unpooling*, el știe exact unde să plaseze valoarea (reconstruită) în harta de *feature*-uri mărită. Plasarea precisă ajută la reconstruirea **granițelor clare** ale obiectelor.

Concept Dificil: Sparse vs. Dense Feature Maps

- **Context:** Când Decoder-ul folosește indicii de *max-pooling* pentru a realiza *Max-unpooling*, el plasează valorile doar în pozițiile de maxim; restul pozițiilor rămân zero. Acest rezultat intermediar este **rar (*sparse*)**.
- Pentru a obține o hartă de *feature*-uri plină și coerentă (o hartă **densă**), SegNet utilizează **straturi convoluționale antrenabile**. Aceste straturi umplu golurile în mod inteligent (rafinează *upsampling*-ul) pe baza *feature*-urilor învățate.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Pierderea Spațială și Soluția):**
  - **CAUZĂ:** Max-pooling-ul oferă invarianță la translație, dar duce la pierderea rezoluției spațiale, afectând detaliul granițelor.
  - **EFECT/SOLUȚIE:** Prin **stocarea indicilor de *max-pooling*** și utilizarea lor în **Max-unpooling**, SegNet reușește să recupereze informația de poziție, permițând o **delineare mai precisă a granițelor**.
- **Conexiunea cu Secțiunea 7.2 (FCN):** SegNet este o îmbunătățire structurală față de FCN. Ambele folosesc o arhitectură Encoder-Decoder complet convoluțională. Totuși, în timp ce FCN folosește convoluția transpusă (eventual combinată ierarhic), SegNet folosește **Max-unpooling bazat pe indici** ca metodă primară de *upsampling* pentru o reconstrucție mai specifică a detaliilor de graniță.

---

5. EXERCITII (Analiză Arhitecturală și Procedurală)

Exercițiul S.1: Analiză Arhitecturală (SegNet vs. VGG-16)

1. Câte straturi convoluționale are rețeaua Encoder a SegNet și de la ce rețea neuronală provin ele?
2. De ce, în SegNet, s-a luat decizia de a elimina straturile complet conectate (Fully Connected Layers) prezente în rețeaua originală VGG-16?

Rezolvare S.1:

1. Rețeaua Encoder a SegNet constă din **13 straturi convoluționale**. Acestea corespund primelor 13 straturi convoluționale din rețeaua **VGG-16**.
2. Straturile complet conectate sunt eliminate în favoarea **reținerii hărților de *feature*-uri de rezoluție mai înaltă** la ieșirea celui mai profund encoder. Acest lucru este necesar deoarece SegNet este o FCN și trebuie să păstreze informațiile spațiale pentru clasificarea la nivel de pixel.

Exercițiul S.2: Procedura Critică (Max-unpooling)

Descrieți rolul indicilor de *max-pooling* stocați în faza de Encoder și cum sunt ei folosiți de către Decoder pentru a produce hărți de *feature*-uri **rare** (sparse).

Rezolvare S.2:

1. **Rolul Indicilor:** În faza de Encoder, în timpul operației de *max-pooling*, se memorează **indicii** sau locațiile unde s-a obținut valoarea maximă în fiecare fereastră de *pooling*.
2. **Producerea Hărților Rare:** Decoder-ul utilizează acești indici memorați pentru a efectua **max-unpooling**. Aceasta înseamnă că valorile *feature*-urilor de intrare (decodate) sunt plasate **doar** în pozițiile spațiale indicate de indici. Deoarece majoritatea pozițiilor din

harta de *feature*-uri mărită nu au corespuns unui maxim în faza de *pooling*, ele rămân inițial cu zero, rezultând astfel **hărți de *feature*-uri rare (*sparse feature maps*)**.

Exercițiul S.3: Analiza Canalelor de Ieșire (Decodare)

Explicați de ce decoder-ul corespunzător primului encoder (cel mai apropiat de imaginea de intrare) este o excepție, producând o hartă de *feature*-uri multi-canal cu **3 canale**.

Rezolvare S.3:

Decoder-ul corespunzător primului encoder produce o hartă de *feature*-uri cu **3 canale** deoarece imaginea de intrare originală (input-ul encoder-ului) este o imagine **RGB** (care are 3 canale de culoare). Această caracteristică este unică în rețeaua SegNet, deoarece celelalte decodere au intrări și ieșiri cu același număr de canale ca și intrarea lor encoder corespunzătoare.

7.4

Absolut! Lecția **7.4 U-Net** descrie o arhitectură de rețea neuronală de top, bazată pe principiile Rețelelor Complet Convoluționale (FCN), dar optimizată pentru a oferi o localizare precisă și o segmentare perfectă a imaginilor mari, fiind deosebit de populară în imagistica medicală.

Ghid de Studiu Complet: 7.4 U-Net

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

U-Net este o arhitectură de rețea neuronală avansată, dezvoltată din ideea de bază a FCN (Rețele Complet Convoluționale), special concepută pentru sarcina de segmentare. Numele "U-Net" provine de la forma sa, care este o structură simetrică, de tip U, compusă dintr-un **Drum Contractant (*Contracting Path* sau Encoder)** și un **Drum Expansiv (*Expansive Path* sau Decoder)**. Scopul U-Net este de a localiza cu precizie pixelii prin combinarea *feature*-urilor de **înaltă rezoluție** din drumul contractant cu ieșirea eșantionată în sus (*upsampled*) din drumul expansiv. Această combinație se realizează prin **conexiuni de tip *skip connection***, similare cu cele din ResNet. U-Net utilizează doar porțiunea validă a fiecărei convoluții, permițând segmentarea fără întreruperi a imaginilor de dimensiuni arbitrar de mari.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Arhitectură în Formă de U:** Rețeaua este compusă dintr-un drum contractant (stânga) și un drum expansiv (dreapta), care este simetric.
2. **Mecanismul Cheie (Skip Connections):** Pentru a îmbunătăți localizarea, *feature*-urile de înaltă rezoluție din drumul contractant sunt concatenate cu ieșirea eșantionată în sus

(upsampled) din drumul expansiv.

3. **Componenta Upsampling:** În drumul expansiv se folosesc operații de **upsampling**, urmate de o **convoluție transpusă 2x2 (*up-convolution*)** care înjumătățește numărul de canale *feature*.
4. **Gestionarea Detaliilor (Canale):** La fiecare pas de *downsampling* (pe drumul contractant), **numărul de canale *feature*** este dublat. Pe drumul expansiv, se folosește un număr mare de canale *feature* pentru a propaga informația contextuală către straturile de rezoluție mai înaltă.
5. **Segmentare Fără Sudură:** Rețeaua folosește doar partea validă a fiecărei convoluții și nu are straturi complet conectate, permițând segmentarea imaginilor de dimensiuni **arbitrar de mari**.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

7.4 U-Net – Arhitectură și Principii

- Ideea principală a U-Net, similară cu FCN (Fully Convolutional Networks), este de a suplimenta o rețea uzuală de contractare (*contracting network*) cu straturi succesive de **upsampling**.
- Straturile de *upsampling* cresc rezoluția ieșirii.
- Arhitectura este simetrică și în formă de U.
- Rețeaua nu are straturi complet conectate (*fully connected layers*).
- În total, rețeaua conține **23 de straturi convoluționale**.

Drumul Contractant (Encoder - Partea Stângă)

- Drumul contractant (partea stângă) urmează arhitectura tipică a unei rețele convoluționale.
- **Structura Repetată:** Constă în aplicarea repetată a **două convoluții de** 3×3 (convoluții fără *padding*), fiecare urmată de o unitate liniară rectificată **ReLU**.
- **Downsampling:** După setul de convoluții și ReLU, urmează o operație de **max-pooling de** 2×2 **cu *stride* 2** pentru sub-eșantionare (*downsampling*).
- **Dublarea Canalelor:** La fiecare pas de *downsampling*, **numărul de canale *feature* este dublat**.

Drumul Expansiv (Decoder - Partea Dreaptă)

- Fiecare pas din drumul expansiv constă în:
  1. O operație de **upsampling** a hărții de *feature*-uri.
  2. Urmează o **convoluție transpusă** 2×2 (numită "*up-convolution*") care **înjumătățește numărul de canale *feature***.

3. O **concatenare** (conexiune *skip connection*, similară cu ResNet) cu harta de *feature*-uri corespunzătoare, **decupată (*cropped*)**, din drumul contractant.
4. Aplicarea a **două convoluții de** 3×3, fiecare urmată de o unitate **ReLU**.

- **Propagarea Contextului:** Partea de *upsampling* are un număr mare de canale *feature*, ceea ce permite rețelei să propage informațiile de context către straturile de rezoluție mai înaltă.

Localizare și Ieșire

- **Combinarea Feature-urilor:** Pentru a localiza cu precizie, *feature*-urile de înaltă rezoluție din drumul contractant sunt combinate cu ieșirea eșantionată în sus (*upsampled output*).
- **Rafinarea Ieșirii:** Un strat convoluțional succesiv poate învăța apoi să asambleze o ieșire mai precisă pe baza acestei informații combinate.
- **Decuparea (*Cropping*):** Decuparea este necesară din cauza **pierderii de pixeli la margini (*loss of border pixels*)** în fiecare operație de convoluție (deoarece se folosesc convoluții fără *padding*).
- **Stratul Final:** La stratul final se folosește o **convoluție** 1×1 pentru a mapa fiecare vector *feature* cu 64 de componente (în exemplul Figura 8) la numărul dorit de clase.
- **Segmentare Fără Sudură:** Strategia permite segmentarea fără sudură a imaginilor de dimensiuni arbitrar de mari, deoarece harta de segmentare de ieșire conține doar pixelii pentru care este disponibil contextul complet în imaginea de intrare.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Skip Connection (Conexiune de Saritură)

- **Explicație Simplificată (Scurtcircuitul de Informație):** Drumul contractant (Encoder) comprimă imaginea, păstrând *feature*-uri semantice de nivel înalt, dar pierzând detalii spațiale (granițele). Drumul expansiv (Decoder) încearcă să reconstruiască detaliile.
- *Asta se întâmplă deoarece:* Conexiunea *skip connection* acționează ca un scurtcircuit: ia *feature*-urile de înaltă rezoluție (detaliile fine ale granițelor) direct de la Encoder și le injectează (le concatenează) înapoi în Decoder la nivelul corespunzător. Aceasta asigură că modelul știe exact **unde** sunt granițele (din *feature*-urile Encoder) în timp ce învață **ce** sunt obiectele (din *feature*-urile Decoder).

Concept Dificil: Rolul Decupării (Cropping)

- **Analogie:** Imaginați-vă că desenați o linie pe o bucată de hârtie fără să apăsați pe margine. La fiecare pas de convoluție **fără *padding***, dimensiunea imaginii se

micșorează. Dacă imaginea de intrare are 572×572, după prima serie de convoluții (fără padding), ea devine 570×570.
- Drumul contractant generează *feature*-uri, dar acestea sunt ușor mai mici din cauza micșorării. Când facem concatenarea, *feature*-urile de pe partea Encoder (care sunt mai mari) trebuie să fie **decupate (*cropped*)** pentru a se potrivi exact cu *feature*-urile de pe partea Decoder (care sunt mai mici).
- Această strategie (fără *padding*) este legată de capacitatea U-Net de a segmenta imagini mari, deoarece se asigură că predicția de segmentare de ieșire (388×388 în Fig. 8) conține doar pixelii pentru care există **context complet** în imaginea de intrare.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Canale Feature):**
  - **CAUZĂ:** U-Net dublează numărul de canale *feature* la fiecare pas de *downsampling*.
  - **EFECT:** Rețeaua poate propaga un număr mare de informații contextuale către straturile de rezoluție mai înaltă, crescând capacitatea de localizare.
- **Conexiunea cu SegNet și FCN:** U-Net este o variantă FCN (folosește *upsampling* pentru a crește rezoluția). Se deosebește de SegNet (Secțiunea 7.3) prin metoda de *upsampling* (utilizează convoluția transpusă, nu *Max-unpooling* bazat pe indici) și prin introducerea conexiunilor *skip connection* de la Encoder la Decoder.

---

5. EXERCITII (Calcul și Analiză Arhitecturală)

Exercițiul U.4: Analiza Arhitecturală U-Net (Stratul Expansiv)

Folosind informațiile din surse (în special Figura 8), descrieți structura completă a unui singur pas pe drumul expansiv (partea dreaptă), menționând ordinea operațiilor și scopul fiecăreia.

Rezolvare U.4:

Un pas tipic pe drumul expansiv (Decoder) constă din:

1. **Upsampling / Up-convolution:** Se realizează o operație de **upsampling** urmată de o **convoluție transpusă de** 2×2 ("*up-convolution*"). Scopul este de a crește dimensiunea spațială și de a **înjumătăți numărul de canale *feature***.
2. **Concatenare (Skip Connection):** Harta de *feature*-uri rezultată este concatenată cu harta de *feature*-uri corespunzătoare (care a fost decupată) din drumul contractant. Scopul este de a furniza **feature-uri de înaltă rezoluție** pentru o localizare precisă.

3. **Rafinare:** Se aplică **două convoluții de** 3×3, fiecare urmată de o unitate **ReLU**. Scopul este de a învăța asamblarea unei ieșiri mai precise bazate pe informațiile combinate și de a rafina *feature*-urile.

Exercițiul U.5: Calculul Canalelor (Drumul Contractant)

Conform Figurii 8, U-Net începe cu o intrare de 1 canal și folosește 64 de canale după primul bloc convoluțional.

1. Determinați numărul de canale *feature* la cel de-al doilea pas de *downsampling* (după al doilea *max-pooling*).
2. Determinați numărul de canale *feature* la cel mai profund nivel al rețelei (vârful literei U), presupunând că sunt 4 blocuri de *downsampling* înainte de a ajunge la acel nivel.

Rezolvare U.5:

1. **Canale la Al Doilea Pas Downsampling:** Drumul contractant dublează numărul de canale la fiecare pas de *downsampling*.
   - Pas 1 (după primul Max-Pooling): 64 de canale.
   - Pas 2 (după al doilea Max-Pooling): $64 \times 2 = \mathbf{128}$ de canale (confirmat de Fig. 8).
2. **Canale la Cel Mai Profund Nivel (Pasul 4):**
   - Pas 1: 64
   - Pas 2: 128
   - Pas 3: $128 \times 2 = 256$
   - Pas 4: $256 \times 2 = \mathbf{512}$ de canale (confirmat de Fig. 8).

Exercițiul U.6: Analiza Dimensională (Convoluții Fără Padding)

Dacă o rețea U-Net primește o imagine de intrare de 100×100 și aplică două straturi de convoluție de 3×3 **fără *padding***, care va fi dimensiunea spațială a hărții de *feature*-uri după aceste două convoluții?

Rezolvare U.6:

Formula de reducere a dimensiunii spațiale pentru o convoluție $k \times k$ fără *padding* este o reducere de $k - 1$ pixeli în fiecare dimensiune (înălțime și lățime).

1. **După Prima Convoluție (3×3):** $100 - (3 - 1) = 100 - 2 = 98 \times 98$.
2. **După A Doua Convoluție (3×3):** $98 - (3 - 1) = 98 - 2 = 96 \times 96$.

Dimensiunea finală va fi $\mathbf{96} \times \mathbf{96}$.

7.5

Lecția **7.5 Dilated Convolution (Convoluția Dilatată sau Atrous)** prezintă o tehnică avansată, crucială în segmentarea semantică, care abordează problema majoră a pierderii

rezoluției spațiale cauzate de operațiile repetitive de *max-pooling* și *striding*. Această metodă oferă o alternativă eficientă la convoluția transpusă, permițând calcularea hărților de *feature*-uri finale la o rezoluție arbitrar de înaltă.

Ghid de Studiu Complet: 7.5 Dilated Convolution

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Convoluția Dilatată (sau *atrous*) este o operație introdusă pentru a îmbunătăți segmentarea semantică, ocolind dezavantajele asociate cu **pierderea rezoluției spațiale** cauzată de straturile consecutive de *max-pooling* și *striding*. Această metodă, dezvoltată inițial pentru transformata wavelet în procesarea semnalelor, permite calcularea răspunsurilor oricărui strat la o **rezoluție dorită**. Convoluția dilatată realizează acest lucru prin inserarea de **zerouri** între valorile consecutive ale filtrului (*kernel*), mărind efectiv **câmpul receptiv** fără a crește numărul de parametri sau costul computațional. Astfel, permite controlul explicit al rezoluției spațiale a hărților de *feature*-uri.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Problema Abordată:** Pierderea semnificativă a rezoluției spațiale (tipic cu un factor de 32) din cauza *max-pooling*-ului și *striding*-ului repetat.
2. **Mecanismul de Bază:** Inserarea a $r-1$ zerouri între valorile filtrului, unde r este **rata de dilatație** (*rate parameter*).
3. **Rata de Dilatație (r):** Parametrul r corespunde *stride*-ului cu care se eșantionează semnalul de intrare. Convoluția standard este un caz special unde $r=1$.
4. **Beneficiu:** Mărește arbitrar **câmpul receptiv efectiv** al filtrului (de la $k \times k$ la k_e) fără a crește numărul de parametri sau operații.
5. **Aplicație (Abordarea Hibridă):** Poate fi folosită pentru a crește densitatea spațială a hărților de *feature*-uri (ex. cu un factor de 4), urmată de interpolare biliniară rapidă pentru a recupera rezoluția originală.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

7.5 Convoluția Dilatată (Dilated Convolution)

Context și Necesitate

- Combinarea repetată a *max-pooling*-ului și *striding*-ului în straturile CNN reduce semnificativ rezoluția spațială a hărților de *feature*-uri (tipic cu un factor de 32 pe fiecare direcție).
- Convoluția transpusă (*transposed convolution*) este o soluție parțială, dar necesită

memorie și timp suplimentar.
- Convoluția dilatată (atrous) este utilizată în schimb; ea a fost dezvoltată inițial pentru calculul eficient al transformatei wavelet.

Definiția și Formula (Semnal 1D)

- Algoritmul permite calcularea răspunsurilor oricărui strat la o rezoluție arbitrară.
- Pentru un semnal de intrare 1D, x_i, și un filtru w_k de lungime K, ieșirea y_i a convoluției dilatate este definită ca:

$$y_i = \sum_{k=1}^{K} x_{i+r \cdot k} w_k$$

- Parametrul ratei r corespunde *stride*-ului cu care se eșantionează semnalul de intrare.
- **Convoluția standard** este un caz special de convoluție dilatată când $r = 1$.

Mecanismul de Dilatare (Inserarea de Zerouri)

- Convoluția dilatată cu rata r introduce $r - 1$ **zerouri** între valorile consecutive ale filtrului (*kernel*).
- Acest lucru mărește efectiv dimensiunea *kernel*-ului de la $k \times k$ la o dimensiune efectivă $k_e = k + (k - 1)(r - 1)$.
- Exemplu 1D (Figura 9): O convoluție standard ($r = 1$) eșantionează intrările consecutiv. O convoluție dilatată cu $r = 2$ (inserând un zero) eșantionează din doi în doi pixeli.

Avantajele și Controlul Câmpului Receptiv

- Deși dimensiunea efectivă a filtrului crește, se ține cont doar de valorile non-zero ale filtrului.
- Astfel, atât **numărul de parametri de filtru, cât și numărul de operații per poziție rămân constante**.
- Convoluția dilatată permite **mărirea arbitrară a câmpului receptiv** al filtrelor la orice strat.
- Aceasta oferă un mecanism eficient pentru a controla câmpul receptiv și pentru a găsi cel mai bun compromis între **localizarea precisă** (câmp receptiv mic) și **asimilarea contextului** (câmp receptiv mare).

Implementarea în Rețelele Profunde (CNN)

- Convoluția dilatată poate fi utilizată în lanțul de straturi al unui CNN pentru a calcula hărțile de *feature*-uri finale la o **rezoluție arbitrar de înaltă**.
- **Exemplu de Dublare a Densității Spațiale:** Pentru a dubla densitatea spațială în VGG-16 sau ResNet-101, se setează *stride*-ul ultimului strat de *pooling* sau convoluțional care scade rezoluția la 1 (pentru a evita decimarea semnalului), și se înlocuiesc toate straturile convoluționale ulterioare cu convoluții *atrous* având **rata** $r = 2$.

- **Abordarea Hibridă Eficientă:** O metodă care echilibrează eficiența și acuratețea este utilizarea convoluției dilatate pentru a crește densitatea *feature map*-urilor cu un factor de **4**, urmată de interpolare biliniară rapidă (cu un factor suplimentar de 8) pentru a recupera rezoluția originală a imaginii.
- Interpolarea biliniară este suficientă în acest context, deoarece hărțile de scor de clasă (*class score maps*), corespunzând probabilităților, sunt destul de **netede (*quite smooth*)**.
- Această abordare hibridă transformă clasificarea imaginilor în extragerea de *feature*-uri dense, fără a necesita învățarea de parametri suplimentari (spre deosebire de convoluția transpusă), ceea ce duce la o antrenare mai rapidă.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Atrous (Dilated) Convolution

- **Analogie (Grătarul de Eșantionare):** O convoluție standard (r=1) rulează un *kernel* mic și compact peste fiecare pixel consecutiv al imaginii. Convoluția dilatată este ca și cum ai folosi același *kernel* mic, dar ai pune un **grătar mare și rar** în fața lui. *Kernel*-ul atinge doar pixelii care trec prin găurile grătarului, sărind peste cei intermediari (de unde și inserarea de zerouri).
- *Asta se întâmplă deoarece:* Această săritură (dilatare) face ca *kernel*-ul să "vadă" o zonă mult mai mare în imaginea de intrare, obținând context pe distanțe lungi, fără a fi nevoie să crești fizic dimensiunea *kernel*-ului sau să adaugi noi ponderi de învățat.

Concept Dificil: Densitate Spațială (Spatial Density)

- **Explicație Simplificată:** *Downsampling*-ul (prin *pooling* sau *striding*) reduce imaginea de intrare. Dacă o imagine 256×256 devine 8×8, densitatea spațială este foarte mică. Convoluția dilatată permite rețelei să calculeze *feature*-uri la o rezoluție mai mare (deci, densitate spațială mai mare), spre deosebire de o rețea standard care, dacă ar fi eliminat *pooling*-ul, ar trebui să folosească *kernel*-uri masive și ar deveni ineficientă.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Dilatare și Câmp Receptiv):**
  - **CAUZĂ:** Inserarea de $r - 1$ zerouri între valorile filtrului.
  - **EFECT:** Mărirea arbitrară a câmpului receptiv al filtrului, permițând asimilarea unui context mai larg, în timp ce numărul de parametri și complexitatea computațională rămân neschimbate.

- **Conexiunea cu Secțiunea 7.1 (Upsampling):** Convoluția dilatată este prezentată ca o alternativă la convoluția transpusă, evitând dezavantajele acesteia (memorie și timp suplimentar). Abordarea hibridă utilizează convoluția dilatată pentru a crește densitatea, dar apelează la interpolarea biliniară (o metodă simplă de *upsampling* din Secțiunea 7.1) pentru a atinge rezoluția finală a imaginii.

---

5. EXERCITII (Calcul și Analiză Dimensională)

Exercițiul D.4: Aplicarea Convoluției Dilatate (1D)

Fie un filtru (*kernel*) 1D $w = [w_1, w_2, w_3]$ de lungime $K = 3$ și un semnal de intrare x. Se aplică o convoluție dilatată cu **rata** $r = 3$ și se calculează ieșirea y_i.

1. Scrieți formula explicită pentru calculul y_i folosind rata $r = 3$.
2. Câte zerouri sunt inserate între valorile consecutive ale filtrului w?
3. Determinați numărul de elemente din semnalul de intrare x care sunt sărite (*skipped*) de la o ponderare la alta în timpul aplicării filtrului.

Rezolvare D.4:

1. **Formula Explicită ($r = 3$):** Folosind formula $y_i = \sum_{k=1}^{K} x_{i+r \cdot k} w_k$ cu $K = 3$ și $r = 3$:

$$y_i = x_{i+3 \cdot 1} w_1 + x_{i+3 \cdot 2} w_2 + x_{i+3 \cdot 3} w_3 = \mathbf{x_{i+3} w_1 + x_{i+6} w_2 + x_{i+9} w_3}$$

2. **Numărul de Zerouri Inserate:** Numărul de zerouri inserate este $r - 1$.

$$\text{erouri} = 3 - 1 = \mathbf{2} \text{ erouri.}$$

3. **Elemente Sărite:** Deoarece rata de dilatație este $r = 3$, se eșantionează din 3 în 3 elemente ale semnalului de intrare. Între două ponderi consecutive (w_1 și w_2, sau w_2 și w_3), se sărit peste 2 elemente. Numărul de elemente sărite este **2**.

Exercițiul D.5: Analiză Dimensională și Eficiență (2D)

Se folosește o convoluție standard 3×3 (unde $k = 3$).

1. Calculați dimensiunea efectivă a filtrului (k_e) dacă se aplică o convoluție dilatată cu rata $r = 4$.
2. Explicați de ce, chiar dacă dimensiunea efectivă a filtrului se mărește semnificativ, complexitatea computațională și numărul de parametri rămân constante.

Rezolvare D.5:

1. **Dimensiunea Efectivă a Filtrului (k_e):** Formula este $k_e = k + (k - 1)(r - 1)$.

$$k_e = 3 + (3 - 1)(4 - 1) = 3 + 2 \times 3 = 3 + 6 = \mathbf{9}$$

(Filtrul 3×3 devine efectiv 9×9).

2. **Constanța Parametrilor și Complexității:** Deși filtrul este extins la 9×9, majoritatea acestor poziții sunt umplute cu **zerouri** (3 zerouri între fiecare valoare). Modelul trebuie să ia în considerare doar valorile de filtru **non-zero** (adică cele 9 ponderi originale). Prin urmare, numărul de parametri (ponderile învățabile) și numărul de operații de înmulțire/adunare per poziție **rămân aceleași** ca pentru un filtru standard 3×3.

7.6

Suntem la lecția **7.6 Conditional Random Fields (CRFs)**, care prezintă o metodă importantă pentru îmbunătățirea preciziei localizării în sarcinile de segmentare semantică, în special în ceea ce privește recuperarea granițelor detaliate ale obiectelor.

Ghid de Studiu Complet: 7.6 Conditional Random Fields (CRFs)

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Lecția 7.6 abordează provocarea inerentă a rețelelor convoluționale (CNN) în segmentarea semantică: un *trade-off* între acuratețea localizării și performanța clasificării. Deși modelele CNN profunde sunt excelente la clasificarea obiectelor (datorită invarianței și câmpurilor receptive largi), hărțile lor de scor (*score maps*) oferă doar o poziție aproximativă și o predicție netedă (*smooth responses*), fiind incapabile să **delimiteze precis granițele obiectelor**. O direcție alternativă pentru a rezolva această problemă de localizare este cuplarea capacității de recunoaștere a CNN-urilor cu acuratețea de localizare la nivel de detaliu oferită de **Conditional Random Fields (CRFs)** complet conectate (*fully connected CRFs*). Aceste CRFs, spre deosebire de variantele tradiționale, sunt capabile să producă rezultate de segmentare semantică precise și să recupereze granițele obiectelor cu un nivel de detaliu care depășește metodele existente.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Problema CNN:** CNN-urile profunde oferă o performanță excelentă de clasificare, dar cedează în fața unei **localizări precise** (granițe netede, nu detaliate).
2. **Soluția Alternativă:** Cuplarea capacității de recunoaștere a CNN-urilor cu precizia de localizare a **CRF-urilor complet conectate**.
3. **Rolul CRF-urilor Tradiționale:** Tradițional, CRFs cu rază scurtă (*short-range*) erau folosite pentru a **netezi hărțile de segmentare zgomotoase**.
4. **Rolul CRF-urilor Complet Conectate:** Acestea depășesc limitările CRF-urilor cu rază scurtă și folosesc o **funcție energetică** compusă din potențiali unari (din CNN) și potențiali de pereche (care asigură uniformitatea etichetelor între pixeli similari).
5. **Mecanismul de Fine-Tuning:** Potențialul de pereche folosește două *kernel*-uri Gaussiene, unde primul *kernel* (*bilateral*) forțează pixelii cu **culoare și poziție similară**

să aibă etichete similare, recuperând astfel detaliile fine.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

7.6 Conditional Random Fields (CRFs)

Provocarea Localizării în CNN-uri

- Există un *trade-off* inerent în CNN-uri între acuratețea localizării și performanța clasificării.
- Modelele mai profunde, cu multiple straturi de *max-pooling*, au câmpuri receptive largi și sunt excelente în clasificarea sarcinilor.
- Totuși, nodurile de nivel superior produc răspunsuri care sunt doar netede (*smooth responses*).
- Hărțile de scor CNN pot prezice prezența și poziția brută (*rough position*) a obiectelor, dar **nu le pot delimita cu adevărat granițele**.
- **Direcțiile anterioare** pentru a rezolva această problemă au fost:
  1. Folosirea informațiilor din multiple straturi ale rețelei convoluționale pentru o estimare mai bună a granițelor obiectelor.
  2. Utilizarea unei reprezentări *super-pixel* care deleagă sarcina de localizare unei metode de segmentare de nivel scăzut (*low-level segmentation method*).

CRFs Tradiționale vs. CRFs Complet Conectate

- **CRFs Tradiționale (Short-range):**
  - Au fost utilizate în mod tradițional pentru a **netezi hărțile de segmentare zgomotoase** (*smooth noisy segmentation maps*).
  - Cuplează de obicei nodurile vecine, favorizând aceleași etichete pentru pixelii proximi spațial.
  - Funcția lor primară era curățarea predicțiilor eronate (*spurious predictions*) ale clasificatorilor slabi.
- **Inadecvarea CRFs Tradiționale:** Hărțile de scor moderne ale arhitecturilor CNN sunt **calitativ diferite** (tipic netede, producând rezultate omogene). Folosirea CRF-urilor cu rază scurtă poate fi dăunătoare în acest regim, deoarece obiectivul este de a **recupera structura locală detaliată**, nu de a netezi și mai mult imaginea.
- **Soluția CRF Complet Conectate (*Fully Connected CRF*):** Acestea depășesc limitările CRF-urilor cu rază scurtă. Ele oferă o acuratețe de localizare la nivel de detaliu (*fine-grained*) care, în combinație cu CNN-urile, produce rezultate precise de segmentare semantică și recuperează granițele obiectelor.

Modelul Funcției Energetice

Modelul CRF complet conectat folosește funcția energetică $(\mathbf{x})$:

$$(\mathbf{x}) = \sum_i {}_i(x_i) + \sum_{ij} {}_{ij}(x_i, x_j)$$

- $\mathbf{x}$ reprezintă **atribuirea etichetelor** (*label assignment*) pentru pixeli.

1. **Potențialul Unar (${}_i(x_i)$):**
   - Este dat de $-\log P(x_i)$, unde $P(x_i)$ este probabilitatea de atribuire a etichetei la pixelul i, așa cum este calculată de **CNN**.
   - Acesta încorporează capacitatea de recunoaștere a CNN-ului.
2. **Potențialul de Pereche (${}_{ij}(x_i, x_j)$):**
   - Are o formă care permite inferența eficientă, chiar dacă conectează **toate perechile de pixeli** (i, j) din imagine (graf complet conectat).
   - Expresia utilizată este:

$${}_{ij}(x_i, x_j) = \mu(x_i, x_j) \left[w_1 \exp\left(-\frac{||\mathbf{p}_i - \mathbf{p}_j||^2}{2\sigma_\alpha^2} - \frac{||\mathbf{l}_i - \mathbf{l}_j||^2}{2\sigma_2} \right) + w_2 \exp\left(-\frac{||\mathbf{p}_i - \mathbf{p}_j||^2}{2\sigma_2} \right) \right]$$

- Funcția $\mu(x_i, x_j)$ este 1 dacă $x_i \neq x_j$ și 0 altfel, penalizând doar nodurile cu **etichete distincte**.

Analiza Kernel-urilor Gaussiene

Potențialul de pereche folosește două *kernel*-uri Gaussiene în spații de *feature*-uri diferite:

1. **Primul Kernel (*Kernel Bilateral*):**
   - Depinde atât de **poziția pixelilor** $(\mathbf{p}_i)$, cât și de **culoarea RGB** $(\mathbf{l}_i)$.
   - Forțează pixelii cu **culoare similară și poziție similară** să aibă etichete similare. Acest *kernel* este esențial pentru recuperarea detaliilor fine ale granițelor.
2. **Al Doilea Kernel:**
   - Depinde doar de **poziția pixelilor** $(\mathbf{p}_i)$.
   - Asigură netezimea (*smoothness*) prin aplicarea proximității spațiale.

- Hiperparametrizarea: σ_α, σ, și σ controlează scara *kernel*-urilor Gaussiene.

Inferența Eficientă

- Acest model este propice pentru inferența probabilistică aproximativă eficientă.
- Actualizările prin transmitere de mesaje (*message passing updates*) sub o aproximare *mean field* pot fi exprimate ca **convoluții Gaussiene** în spațiul bilateral.
- Algoritmii de filtrare de înaltă dimensiune accelerează semnificativ acest calcul, rezultând un algoritm **foarte rapid în practică**.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: Potențial Unar (Unary Potential)

- **Explicație Simplificată (Încrederea CNN):** Potențialul unar este practic o măsură a cât de **încrezător** este clasificatorul CNN (fără nicio corecție contextuală) că un anumit pixel aparține unei anumite clase. Dacă CNN-ul este foarte încrezător ($P(x_i)$ mare), potențialul unar ($\log P(x_i)$) va fi mic, ceea ce minimizează energia totală.

Concept Dificil: Graf Complet Conectat în CRF

- **Analogie (Influența la Distanță):** CRF-urile tradiționale luau în considerare doar vecinii direcți (o mică comunitate). CRF-urile complet conectate iau în considerare **toate** perechile de pixeli din imagine (toți pixelii influențează toți ceilalți). Acest lucru era anterior prohibitiv din punct de vedere computațional.
- *Asta se întâmplă deoarece:* Deși teoretic toți pixelii sunt conectați, Potențialul de Pereche folosește *kernel*-uri Gaussienne. Aceste *kernel*-uri asigură că pixelii apropiați (atât spațial, cât și cromatic) au o influență mare (penalizare mare dacă au etichete diferite), în timp ce pixelii foarte îndepărtați au o influență aproape de zero, permițând inferența eficientă și rapidă prin algoritmi de filtrare de înaltă dimensiune.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Regimul de Segmentare Semantică):**
  - **CAUZĂ:** Hărțile de scor CNN sunt netede (*smooth*) și omogene.
  - **EFECT:** CRFs cu rază scurtă devin dăunătoare (deoarece le-ar netezi și mai mult).
  - **SOLUȚIE:** Necesitatea de a utiliza CRF-uri complet conectate cu *kernel*-uri sensibile la contrast (bilateral) pentru a **recupera detaliile structurale locale**.
- **Conexiunea cu Capitolul 7 (Segmentare):** CRF-urile sunt un strat de **post-procesare** care ia ieșirea brută, netedă (dar semantic corectă) a rețelelor de segmentare (precum FCN, SegNet sau U-Net, în stratul final înainte de softmax) și o rafinează pentru a corespunde granițelor vizuale clare din imaginea de intrare.

---

5. EXERCITII (Analiză și Formulă)

Exercițiul R.1: Analiza Funcției Energetice CRF

Identificați cele două componente principale ale funcției energetice $(\mathbf{x})$ a CRF-urilor complet conectate și explicați sursa informației pe care se bazează fiecare componentă.

Rezolvare R.1:

Funcția energetică $(\mathbf{x})$ este compusă din:

1. **Potențialul Unar ($\sum_i {}_i(x_i)$):** Măsoară costul atribuirii unei anumite etichete x_i pixelului i. Sursa informației este **probabilitatea de atribuire a etichetei** $P(x_i)$, calculată direct de **CNN**.
2. **Potențialul de Pereche ($\sum_{ij} {}_{ij}(x_i, x_j)$):** Măsoară costul atribuirii etichetelor x_i și x_j perechii de pixeli (i, j). Sursa informației este **similitudinea vizuală și spațială** dintre pixeli (culoare RGB și poziție), asigurând că pixelii similari vizual primesc aceeași etichetă.

Exercițiul R.2: Rolul Kernel-ului Bilateral

Potențialul de pereche folosește un *kernel bilateral* (primul *kernel* Gaussian). Care sunt cele două tipuri de informații de *feature* pe care se bazează acest *kernel*, și care este scopul său principal în contextul recuperării granițelor?

Rezolvare R.2:

1. **Informațiile de Feature Utilizate:**
   - **Poziția Pixelilor** $(\mathbf{p}_i)$.
   - **Culoarea RGB** $(\mathbf{l}_i)$.
2. **Scopul Principal:** Acest *kernel* forțează pixelii care au **culoare similară și poziție similară** să aibă etichete semantice similare. Prin combinarea informațiilor despre culoare și poziție, CRF-ul poate "sări" peste marginile estimate greșit de CNN și poate adera la granițele vizuale clare din imaginea originală (de exemplu, o linie cromatică clară), realizând astfel **recuperarea detaliilor fine**.

Exercițiul R.3: Analiza Vizuală a Îmbunătățirii (Figura 12)

Analizați vizual Figura 12 (aplicată pe un avion).

1. Descrieți calitativ ieșirea CNN înainte de aplicarea CRF.
2. Descrieți cum evoluează harta de credință (*belief map*) după 10 iterații ale CRF (CRF Iteration 10) comparativ cu ieșirea CNN.

Rezolvare R.3:

1. **Ieșirea CNN (Prima Rând, Coloana 2):** Ieșirea CNN (harta de scor) este **netedă** (*smooth*) și indică prezența și poziția aproximativă a obiectului (avionul), dar **nu reușește să delimiteze granițele clare**. Forma este difuză.
2. **Evoluția după 10 Iterații:** După 10 iterații, harta de credință (rândul 2, coloana 5) arată că CRF-ul a **rafinat masca de segmentare**. CRF-ul a recuperat structura detaliată, iar granițele avionului sunt acum **clar delimitate și fidele** granițelor *ground-truth* (G.T.), în ciuda faptului că intrarea CNN inițială era netedă.

7.7

Lecția **7.7 DeepLab** prezintă o arhitectură avansată și de referință în segmentarea semantică, care integrează eficient trei tehnici majore: **Rețelele Complet Convoluționale (FCN)**, **Convoluția Dilatată (*Atrous*)** pentru controlul rezoluției și **Conditional Random Fields (CRF)** pentru rafinarea granițelor. Inovația cheie a DeepLab este metoda **Atrous Spatial Pyramid Pooling (ASPP)**, care îmbunătățește capacitatea rețelei de a gestiona obiecte de diferite scări.

Ghid de Studiu Complet: 7.7 DeepLab

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

Modelul **DeepLab** este o arhitectură de segmentare semantică bazată pe rețele convoluționale profunde (precum VGG-16 sau ResNet-101), care sunt re-intenționate (*re-purposed*) pentru sarcinile de segmentare. DeepLab combină puterea de recunoaștere a CNN-urilor cu o localizare precisă, abordând două provocări majore: pierderea rezoluției spațiale și necesitatea de a modela obiecte la scări multiple. Rezoluția spațială este controlată prin **convoluții dilatate** care cresc densitatea *feature map*-urilor (de la 32x la 8x sub-eșantionare). Abordarea scalelor multiple este realizată prin **Atrous Spatial Pyramid Pooling (ASPP)**, care folosește multiple filtre dilatate, paralele, cu rate diferite de eșantionare. În cele din urmă, rezultatul este rafinat printr-un **CRF complet conectat** pentru a recupera granițele obiectelor.

Care sunt cele 4 idei principale pe care trebuie neapărat să le rețin?

1. **Fundament:** DeepLab este o rețea convoluțională profundă transformată într-o **rețea complet convoluțională (FCN)**.
2. **Controlul Rezoluției:** Rezoluția *feature map*-urilor este crescută de la o sub-eșantionare de $32\times$ la o sub-eșantionare de $8\times$ folosind **convoluții dilatate**.
3. **Detectarea Multi-Scală:** Se utilizează **Atrous Spatial Pyramid Pooling (ASPP)**, o schemă care folosește straturi convoluționale dilatate paralele, cu rate $r = 6, 12, 18, 24$, pentru a capta *feature*-uri la scări diferite.
4. **Rafinarea Ieșirii:** Harta de scor (score map) este mărită (*upsampled*) prin **interpolare biliniară** și apoi rafinată folosind un **CRF complet conectat** pentru a îmbunătăți granițele obiectelor.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

7.7 DeepLab – Procesul de Re-intenționare (Re-purposing)

- DeepLab folosește o rețea neuronală convoluțională profundă (VGG-16 sau ResNet-101) antrenată pe sarcina de clasificare a imaginii și o re-intenționează pentru segmentarea semantică.
- Această re-intenționare implică două etape cheie:
  1. **Conversia în FCN:** Transformarea tuturor straturilor complet conectate în **straturi convoluționale**.
  2. **Creșterea Rezoluției Feature-urilor:** Rezoluția *feature*-urilor este mărită prin straturi convoluționale dilatate, permițând calcularea hărților de *feature*-uri **la fiecare 8 pixeli** în loc de fiecare 32 de pixeli ca în rețeaua originală.

7.7.1 Controlul Rezoluției și Antrenarea

- **Creșterea Densității Spațiale:** Pentru a **cvadrupla (de 4 ori)** densitatea spațială a răspunsurilor *feature*-urilor în rețelele VGG-16 sau ResNet-101, se setează *stride*-ul ultimelor două straturi de *pooling* sau convoluționale care scad rezoluția la 1, prevenind astfel decimarea semnalului.
- Toate straturile convoluționale ulterioare sunt înlocuite cu **convoluții dilatate având rata** $r = 2$.
- **Pierderea (Loss Function):** Funcția de pierdere este suma termenilor de entropie încrucișată (*cross-entropy*) pentru fiecare poziție spațială în harta de ieșire a CNN (care este sub-eșantionată de 8 ori față de imaginea originală).
- Toate pozițiile și etichetele sunt ponderate egal în funcția de pierdere totală (cu excepția pixelilor neetichetați care sunt ignorați).

7.7.2 Atrous Spatial Pyramid Pooling (ASPP)

- CNN-urile au abilitatea de a reprezenta implicit scara obiectelor, dar luarea explicită în considerare a scalei îmbunătățește performanța.
- DeepLab implementează ASPP, inspirat de succesul metodei R-CNN de *spatial pyramid pooling*.
- **Mecanismul ASPP (Figura 13/15):** ASPP utilizează **multiple straturi convoluționale dilatate (atrous) paralele** cu **rate de eșantionare diferite**.
  - *Feature*-urile extrase pentru fiecare rată de eșantionare sunt procesate în ramuri separate și apoi **fuzionate** pentru a genera rezultatul final.
  - În modelul ASPP, se folosesc **patru ramuri** de 3×3 convoluții, cu rate de dilatație $r = 6, 12, 18, 24$.
  - Exemplu: DeepLab-ASPP (Figura 15b) folosește ramuri paralele fc6-fc7-fc8, aplicând rate diferite r în stratul fc6. Ramura LargeFOV (Figura 15a) utilizează o singură ramură cu $r = 12$.

7.7.3 Post-Procesare și Rafinare (CRF)

- Harta de scor (*score map*) calculată de CNN (sub-eșantionată de 8x) este apoi mărită (*upsampled*) la rezoluția imaginii originale.

- Acest *upsampling* se realizează prin **interpolare biliniară** cu un factor de 8.
- Harta de *feature*-uri mărită este apoi folosită ca intrare pentru un **CRF complet conectat**, care **rafinează rezultatele de segmentare** și captează mai bine granițele obiectelor.
- Etapele de antrenare CNN și CRF sunt **decuplate** (separate), presupunând că termenii unari CNN sunt ficși atunci când sunt setați parametrii CRF.

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Concept Dificil: ASPP (Atrous Spatial Pyramid Pooling)

- **Analogie (Vederea la Scări Multiple):** ASPP rezolvă problema recunoașterii obiectelor mari și mici simultan. Imaginați-vă că un filtru normal (rata $r = 1$) are un câmp vizual restrâns. ASPP oferă patru perechi de ochelari:
  1. Ochelari normali ($r = 6$) – Vede detalii fine și obiecte mici.
  2. Ochelari tele-obiectiv mediu ($r = 12$) – Vede obiecte de mărime medie.
  3. Ochelari tele-obiectiv larg ($r = 18$) – Vede obiecte mari și context.
  4. Ochelari ultra-largi ($r = 24$) – Vede contextul extins.
- *Asta se întâmplă deoarece:* Folosind filtre dilatate paralele cu rate diferite, rețeaua poate extrage simultan informații contextuale relevante la **scări multiple** din aceeași hartă de *feature*-uri.

Concept Dificil: De la $32\times$ la $8\times$ Sub-eșantionare

- **Context:** Rețelele CNN precum VGG-16, prin operații succesive de *pooling*, reduc rezoluția imaginii de intrare de $32\times$ (adică, un pixel de *feature map* reprezintă o zonă de 32×32 din imaginea originală).
- **DeepLab:** Pentru a îmbunătăți localizarea, DeepLab **cvadruplează densitatea spațială**. Acest lucru este realizat prin forțarea ultimelor straturi de *pooling* (sau convoluționale) să aibă *stride* 1 și introducerea convoluției dilatate cu $r = 2$. Rezultatul este că un pixel de *feature map* reprezintă doar o zonă de 8×8 din imaginea originală, îmbunătățind precizia spațială.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Conexiunea cu Secțiunea 7.5 (Dilated Convolution):** DeepLab este o aplicație majoră a convoluției dilatate (atrous), demonstrând cum această tehnică permite calcularea hărților de *feature*-uri la o rezoluție arbitrar de înaltă, în loc de a pierde rezoluția cu factorul standard de 32.

- **Conexiunea cu Secțiunea 7.6 (CRF):** DeepLab utilizează CRF-ul complet conectat ca un strat de post-procesare esențial pentru a lua harta de scor *coarse* (netedă, dar semantic corectă) și a o rafina la **granițe precise** folosind informații despre culoare și poziție.
- **Conexiunea cu R-CNN:** ASPP este inspirat de succesul metodei R-CNN de *spatial pyramid pooling*.

---

5. EXERCITII (Calcul și Analiză Arhitecturală)

Exercițiul D.6: Analiza Dimensională a Upsampling-ului

Un model DeepLab prelucrează o imagine de intrare de 1024×1024 pixeli.

1. Determinați dimensiunea spațială a hărții de *feature*-uri (score map) calculată de CNN-ul DeepLab înainte de interpolarea biliniară.
2. Care este factorul de *upsampling* necesar (prin interpolare biliniară) pentru a aduce harta de scor la rezoluția imaginii originale?

Rezolvare D.6:

1. **Dimensiunea Hărții de Scor:** DeepLab folosește o sub-eșantionare totală de $8\times$ față de imaginea originală.

$$\text{imensiune Score ap} = \frac{1024}{8} \times \frac{1024}{8} = \mathbf{128 \times 128}$$

2. **Factorul de Upsampling:** Harta de scor trebuie să ajungă la rezoluția originală, ceea ce implică un *upsampling* cu factorul rămas, care este **8**.

Exercițiul D.7: Analiza ASPP (Câmp Receptiv Efectiv)

În modelul DeepLab-ASPP, se folosește un *kernel* de convoluție 3×3. Se aplică o rată de dilatație $r = 18$ pe una dintre ramurile paralele.

1. Calculați dimensiunea efectivă (k_e) a acestui *kernel* dilatat.
2. Explicați rolul ratei $r = 18$ în comparație cu o rată $r = 6$ în contextul ASPP.

Rezolvare D.7:

1. **Dimensiunea Efectivă a Filtrului (k_e):** Formula este $k_e = k + (k-1)(r-1)$. Unde $k = 3$ și $r = 18$.

$$k_e = 3 + (3-1)(18-1) = 3 + 2 \times 17 = 3 + 34 = \mathbf{37}$$

(Filtrul 3×3 devine efectiv 37×37).

2. **Rolul Ratei $r = 18$ vs. $r = 6$:** Rata $r = 18$ creează un câmp receptiv efectiv mult mai mare decât rata $r = 6$. ASPP utilizează această diferență pentru a exploata **feature-uri multi-scări**. În timp ce $r = 6$ captează informații pe o rază relativ mică (obiecte mici), $r = 18$ captează un context mult mai extins și este esențial pentru detectarea și segmentarea obiectelor **mari**.

Exercițiul D.8: Analiză Teoretică (CRF)

Explicați de ce etapele de antrenare CNN și CRF sunt **decuplate** (separate) în arhitectura DeepLab și care este ipoteza cheie făcută în timpul antrenamentului CRF.

Rezolvare D.8:

Etapele de antrenare CNN și CRF sunt decuplate deoarece CRF-ul acționează ca un strat de **post-procesare** pentru a rafina granițele. CNN este antrenat pentru a oferi hărți de scor semantic corecte (*coarse score maps*).

Ipoteza cheie este că termenii unari CNN (care reprezintă încrederea rețelei) sunt **fixați** atunci când sunt setați parametrii CRF. Aceasta permite CRF-ului să se concentreze strict pe învățarea modului optim de a **îmbunătăți localizarea fină** pe baza culorii și poziției, fără a permite *feedback*-ului de la CRF să modifice ponderile complexe ale rețelei CNN.

7.8

Acționez în calitate de profesor universitar expert și tutore răbdător pentru a vă ghida prin lecția **7.8 DeepLabv3**, o arhitectură de vârf în segmentarea semantică, care rafinează și mai mult tehnicile introduse în versiunile anterioare DeepLab.

Ghid de Studiu Complet: 7.8 DeepLabv3

1. 🎯 SINTEZA LECȚIEI (THE BIG PICTURE)

DeepLabv3 reprezintă o îmbunătățire majoră a arhitecturii DeepLab anterioare, axându-se pe controlul explicit al **densității spațiale** a *feature map*-urilor și pe capturarea informațiilor de context pe distanțe lungi. Modelul introduce module de convoluție dilatată (atrous) aranjate în **cascadă** și un mecanism Multi-Grid pentru a aplica rate de dilatație ierarhice. Cea mai importantă inovație este revizuirea modulului **Atrous Spatial Pyramid Pooling (ASPP)** prin includerea **normalizării pe batch** și prin adoptarea **feature-urilor la nivel de imagine** pentru a incorpora contextul global, rezolvând astfel problema pierderii de ponderi valide la rate mari de dilatație.

Care sunt cele 5 idei principale pe care trebuie neapărat să le rețin?

1. **Definiția `output_stride`:** Raportul dintre rezoluția spațială a imaginii de intrare și rezoluția de ieșire finală.

2. **Controlul Densității Spațiale:** Creșterea densității spațiale de la $\mathbf{output_{stride} = 32}$ (clasificare) la $\mathbf{output_{stride} = 8}$ sau $\mathbf{16}$ (segmentare) se face prin setarea *stride*-ului la 1 și utilizarea convoluției dilatate cu **rata** $r = 2$.
3. **Arhitectura Cascadată:** DeepLabv3 utilizează blocuri ResNet (replicate ale *block4*, numite *block5, block6, block7*) aranjate în cascadă, cu convoluție dilatată.
4. **Multi-Grid:** Se folosesc rate de dilatație ierarhice definite ca $\mathbf{Multi_{Grid} = (r_1, r_2, r_3)}$ în interiorul blocurilor. Rata finală este egală cu înmulțirea ratei unitare cu rata blocului corespunzător.
5. **ASPP Îmbunătățit:** Modulul ASPP revizuit include **normalizarea pe batch (BN)** și **feature-uri la nivel de imagine (*image-level features*)** pentru a capta informații contextuale globale.

---

2. 📖 REZUMAT DETALIAT (PE CAPITOLE/SECȚIUNI)

7.8.1 Controlul Rezoluției și `output_stride`

- **Definiție:** `output_stride` denotă raportul dintre rezoluția spațială a imaginii de intrare și rezoluția de ieșire finală.
- **CNN-uri Standard:** Pentru CNN-urile folosite în clasificarea imaginilor, hărțile de *feature*-uri finale (înainte de straturile complet conectate sau *global pooling*) sunt de **32 de ori mai mici** decât dimensiunea imaginii de intrare, deci **output_stride = 32**.
- **Creșterea Densității Spațiale:** Pentru a cvadrupla densitatea spațială a răspunsurilor *feature*-urilor (adică, $\mathbf{output_{stride} = 8}$), se modifică *stride*-urile ultimelor două straturi de *pooling* sau convoluționale care scad rezoluția, setându-le la **1** pentru a evita decimarea semnalului.
- **Înlocuire:** Toate straturile convoluționale ulterioare sunt înlocuite cu **convoluții dilatate** având **rata** $r = 2$.

7.8.2 Module Cascadate și Decimarea Detaliilor

- DeepLabv3 utilizează module cu convoluție dilatată dispuse în **cascadă**.
- **Implementare:** Mai multe copii ale ultimului bloc ResNet (numit *block4*) sunt duplicate și aranjate în cascadă, denumite *block5, block6, block7*, similar cu ResNet-ul original.
- **Problema Stride-ului:** În ResNet-ul original, ultimul strat convoluțional din blocurile ulterioare conține *stride* 2. Această decimare consecutivă ajută la capturarea informațiilor pe distanțe lungi în blocurile mai profunde, dar este **dăunătoare pentru segmentarea semantică**, deoarece informațiile de detaliu sunt decimate.
- **Soluția Cascadată (Figura 16):** Atrous convolution cu rata $r > 1$ este aplicată după *block3* atunci când $output_stride = 16$.

7.8.3 Metoda Multi-Grid

- DeepLabv3 adoptă o metodă inspirată de *multi-grid* (ierarhie de grile de dimensiuni diferite) pentru a aplica **rate de dilatație diferite** în interiorul blocurilor ResNet.
- **Definiție Multi-Grid:** Se definește $\mathbf{Multi_{Grid}} = (\mathbf{r_1}, \mathbf{r_2}, \mathbf{r_3})$ ca fiind ratele unitare pentru cele trei straturi convoluționale din interiorul blocurilor (block4 până la block7).
- **Calculul Ratei Finale:** Rata de dilatație finală pentru un strat convoluțional este egală cu **înmulțirea ratei unitare** și a **ratei blocului corespunzător**.
- **Exemplu Numeric:** Pentru $output_stride = 16$ și $Multi_Grid = (1, 2, 4)$, cele trei convoluții din *block4* vor avea rate finale de $2 \cdot (1, 2, 4) = \mathbf{(2, 4, 8)}$.

7.8.4 Atrous Spatial Pyramid Pooling (ASPP) Îmbunătățit

- DeepLabv3 revizuiește modulul ASPP, care utilizează patru convoluții dilatate paralele cu rate de dilatație diferite, aplicate pe harta de *feature*-uri.
- **BN Inclus:** Spre deosebire de versiunea anterioară DeepLab, **normalizarea pe batch (*batch normalization*)** este inclusă în ASPP.
- **Problema Ratelor Mari:** S-a descoperit că, pe măsură ce rata de eșantionare devine mai mare, numărul de ponderi de filtru valide aplicate regiunii de *feature* valide devine mai mic (în loc de zerouri *padded*).
- **Soluția (Global Context):** Pentru a depăși această problemă și a încorpora informații de **context global** în model, DeepLabv3 adoptă **feature-uri la nivel de imagine (*image-level features*)**.
  - Aceasta implică aplicarea **global average pooling** pe ultima hartă de *feature*-uri a modelului.
  - Rezultatul este alimentat unei convoluții 1×1 cu 256 de filtre (plus BN) și apoi eșantionat în sus biliniar (*bilinearly upsample*) la dimensiunea spațială dorită.
- **Componența ASPP Îmbunătățit (Figura 17):**
  - O convoluție 1×1 și trei convoluții 3×3 cu rate $r = \mathbf{(6, 12, 18)}$ (când $output_stride = 16$), toate cu 256 de filtre și BN.
  - **Feature-urile la nivel de imagine**.
- Toate *feature*-urile din aceste ramuri sunt concatenate și trec printr-o altă convoluție 1×1 (cu 256 de filtre și BN) înainte de stratul final 1×1 care generează *logits* (ieșirea finală).

---

3. 💡 EXPLICAȚII SUPLIMENTARE ȘI CLARIFICĂRI (IMPORTANT)

Termen Tehnic: output_stride

- **Explicație Simplificată (Scara Hărții):** `output_stride` este scara la care harta de *feature*-uri finală (sau harta de scor) se raportează la imaginea de intrare. Dacă $output_stride = 32$, harta de *feature*-uri este foarte mică (scădere masivă de rezoluție).

Dacă se dorește o localizare mai bună, trebuie să se reducă acest raport (ex. $output_stride = 8$), ceea ce înseamnă că fiecare pixel din harta de *feature*-uri reprezintă o zonă de 8×8 din imaginea originală.

Concept Dificil: Multi-Grid (Ierarhia Ratelor de Dilatație)

- **Analogie (Echipa de Cercetași):** Multi-Grid este un mod de a face ca blocurile ResNet să fie mai eficiente în capturarea contextului la scări diferite. Într-un bloc (ex. Block4), în loc să folosească aceeași rată de dilatație pentru toate cele trei sub-convoluții, se folosește o ierarhie. Imaginați-vă trei cercetași în același bloc: unul se uită la detalii apropiate (r_1), altul la distanță medie (r_2), și al tretelea la distanță mare (r_3). Acest lucru oferă rețelei o perspectivă mult mai nuanțată asupra contextului local la nivel de bloc.

Clarificare Suplimentară: Image-Level Features

- **Context Explanativ:** S-a observat că ratele mari de dilatație (necesare pentru ASPP) lasă multe "găuri" în regiunile valide, reducând ponderea efectivă a filtrelor. Pentru a compensa această pierdere de informații și a obține o înțelegere generală a scenei (context global), DeepLabv3 adaugă *feature*-uri la nivel de imagine. **Global Average Pooling** comprimă întreaga hartă de *feature*-uri într-un vector compact (rezumatul global al imaginii), care este apoi reintrodus în proces, oferind informații despre "ce" este imaginea în ansamblul ei, nu doar ce se întâmplă local.

---

4. 🔗 CONEXIUNI ȘI IMPLICAȚII

- **Relația Cauză-Efect (Consecutive Striding):**
  - **CAUZĂ:** Striding-ul consecutiv în blocurile ResNet (ex. *block4, block5* etc., din rețeaua originală) decimează informațiile de detaliu.
  - **EFECT:** Această decimare este dăunătoare pentru segmentarea semantică.
  - **SOLUȚIE:** Utilizarea convoluției dilatate în cascadă și Multi-Grid pentru a reține rezoluția spațială înaltă, în timp ce se mărește câmpul receptiv.
- **Conexiunea cu Secțiunea 7.7 (DeepLabv1/v2):** DeepLabv3 consolidează utilizarea convoluției dilatate, dar adaugă două îmbunătățiri critice: integrarea BN în ASPP și adăugarea ramurii de *image-level features* pentru contextul global.

---

5. EXERCITII (Calcul și Analiză)

Exercițiul D3.1: Calculul Ratei Finale Multi-Grid

Presupuneți că arhitectura DeepLabv3 folosește blocuri cascadate, iar pentru blocurile ResNet *block5* (care are rata blocului de 4), se definește $Multi_Grid = (1, 3, 5)$.

1. Calculați ratele de dilatație finale pentru cele trei straturi convoluționale din interiorul *block5*.
2. Dacă $output_stride$ este 16, care este rata blocului pentru *block4* (primul bloc cascadat)?

Rezolvare D3.1:

1. **Ratele Finale pentru Block5:** Rata finală = Rata unitară $\times$ Rata blocului (care este 4 pentru block5).

$$Rata_1 = 1 \times 4 = \mathbf{4}$$
$$Rata_2 = 3 \times 4 = \mathbf{12}$$
$$Rata_3 = 5 \times 4 = \mathbf{20}$$

2. **Rata Blocului pentru Block4 ($output_stride = 16$):** În exemplul dat de sursă, pentru $output_stride = 16$, rata blocului pentru *block4* este **2**.

Exercițiul D3.2: Analiza Output Stride și Rezoluția

Se consideră o imagine de intrare de 512×512.

1. Dacă s-ar folosi un CNN standard pentru clasificare (fără modificări), care ar fi dimensiunea *feature map*-urilor finale, având în vedere că $output_stride = 32$?
2. Dacă se folosește DeepLabv3 cu $output_stride = 16$, care este dimensiunea *feature map*-urilor finale (harta de scor) înainte de orice *upsampling* biliniar?

Rezolvare D3.2:

1. **Dimensiunea cu $output_stride = 32$:**

$$512/32 = \mathbf{16} \times \mathbf{16}$$

2. **Dimensiunea cu $output_stride = 16$:**

$$512/16 = \mathbf{32} \times \mathbf{32}$$

Exercițiul D3.3: Analiză Conceptuală ASPP Îmbunătățit

Explicați de ce includerea feature-urilor la nivel de imagine în ASPP, realizată prin *Global Average Pooling*, este crucială pentru performanța DeepLabv3 și ce problemă încearcă să rezolve această componentă.

Rezolvare D3.3:

Feature-urile la nivel de imagine rezolvă două probleme majore:

1. **Pierderea Ponderilor Valide:** La rate mari de dilatație (necesare pentru ASPP), un număr tot mai mic de ponderi de filtru valide (non-zero) sunt aplicate regiunii de *feature* valide.

2. **Incorporarea Contextului Global:** Prin aplicarea *Global Average Pooling* pe ultima hartă de *feature*-uri, DeepLabv3 obține un rezumat compact al întregii imagini (context global). Această informație globală este crucială pentru a ajuta rețeaua să înțeleagă întregul cadru al scenei, compensând limitările vederii locale induse de ratele mari de dilatație.

7.9

Înțeleg necesitatea unui set de exerciții complexe, tipice unui examen final, care să acopere integral subiectele Segmentării Semantice (de la concepte de bază până la arhitecturile DeepLabv3).

Voi structura exercițiile în trei părți: **I. Concepte de Bază și Diferențiere**, **II. Analiză Arhitecturală Comparativă (SegNet, U-Net, DeepLab)** și **III. Exerciții de Calcul Aplicat (Upsampling & Convoluție Dilatată)**.

EXERCIȚII FINALE: VIZIUNE COMPUTERIZATĂ - SEGMENTARE SEMANTICĂ

I. Concepte de Bază și Diferențiere

Întrebarea I.1 (Segmentare Semantică vs. Detecție Obiecte):

Explicați diferența fundamentală dintre sarcina de **Detecție a Obiectelor** (Capitolul 6) și **Segmentarea Semantică** (Capitolul 7). Cum se manifestă diferența în precizia granițelor?

Răspuns I.1:

Detecția Obiectelor folosește **cutii de delimitare dreptunghiulare** (*rectangular bounding boxes*) pentru a eticheta și prezice obiecte în imagini,. Segmentarea Semantică, spre deosebire de detecție, se concentrează pe împărțirea imaginii în regiuni aparținând diferitelor clase semantice. Aceasta recunoaște și înțelege conținutul imaginilor la **nivel de pixel**. În comparație cu detecția obiectelor, granițele regiunilor etichetate la nivel de pixel în segmentarea semantică sunt **evident mai detaliate (*obviously more fine-grained*)**,.

Întrebarea I.2 (Instanțe și Stuff vs. Things):

Definiți și diferențiați **Segmentarea Instanțelor** de **Segmentarea Panoptică**. Care este distincția făcută în Segmentarea Panoptică între obiectele nenumărabile și cele numărabile?,,,

Răspuns I.2:

Segmentarea Instanțelor (cunoscută și ca *simultaneous detection and segmentation*) studiază modul de recunoaștere a regiunilor la nivel de pixel pentru **fiecare instanță de**

obiect dintr-o imagine,. Spre deosebire de segmentarea semantică simplă, segmentarea instanțelor trebuie să distingă nu doar semantica, ci și **diferitele instanțe de obiect** (de exemplu, cui aparține un pixel dacă sunt două vaci),,,.

Segmentarea Panoptică combină ambele sarcini: oferă etichete pentru obiecte **nenumărabile** numite "stuff" (precum cerul sau iarba, similar segmentării semantice) și diferențiază pixelii provenind din **instanțe diferite** ale aceleiași clase (obiecte numărabile numite "things", precum vacile), similar segmentării instanțelor,.

Întrebarea I.3 (FCN și Upsampling):

Care este necesitatea esențială a operației de **Upsampling** (eșantionare în sus) în arhitecturile de segmentare semantică, cum ar fi FCN (Fully Convolutional Network)?,,,

Răspuns I.3:

Straturile CNN obișnuite (convoluționale și *pooling*) reduc în mod tipic dimensiunile spațiale ale intrării (downsample),. Deoarece segmentarea semantică clasifică la **nivel de pixel**,, este vital ca dimensiunile spațiale ale intrării și ieșirii să fie **aceleași**,. FCN realizează acest lucru prin transformarea înălțimii și lățimii hărților de *feature*-uri intermediare înapoi la cele ale imaginii de intrare, utilizând stratul de **convoluție transpusă**,. Astfel, dimensiunea canalului la un pixel de ieșire reține rezultatele clasificării pentru pixelul de intrare din aceeași poziție spațială,,,.

II. Analiză Arhitecturală Comparativă (SegNet, U-Net, DeepLab)

Întrebarea II.1 (SegNet vs. FCN):

Care este mecanismul principal de *upsampling* utilizat în arhitectura **SegNet** (Encoder-Decoder) și ce informație esențială este stocată de Encoder pentru a permite o delineare precisă a granițelor în Decoder?,,,

Răspuns II.1:

SegNet realizează *upsampling* (eșantionare în sus) pas cu pas, utilizând operația de **Max-unpooling**,. Pentru a combate pierderea rezoluției spațiale cauzată de *max-pooling* (care este dăunătoare pentru delinearea granițelor),, Encoder-ul memorează doar **indicii de *max-pooling*** (adică, locațiile valorii maxime în fiecare fereastră de *pooling*),. Decoder-ul utilizează acești indici memorați pentru a efectua *Max-unpooling*-ul,, producând inițial hărți de *feature*-uri **rare (*sparse*)**,.

Întrebarea II.2 (U-Net și Skip Connections):

Explicați rolul conexiunilor de tip **Skip Connection** (concatenare) în arhitectura **U-Net** și de ce se folosește un număr mare de canale *feature* pe drumul expansiv (Decoder).,

Răspuns II.2:

U-Net utilizează conexiuni de tip **Skip Connection** pentru a combina **feature-urile de înaltă rezoluție** din drumul contractant (*contracting path*) cu ieșirea eșantionată în sus (*upsampled output*) din drumul expansiv (*expansive path*),. Această combinare este crucială pentru a obține o **localizare precisă** a segmentării.

Un număr mare de canale *feature* este folosit pe drumul expansiv pentru a permite rețelei să **propage informațiile de context** către straturile de rezoluție mai înaltă.

Întrebarea II.3 (DeepLab și CRFs):

DeepLab folosește CRFs complet conectate ca un strat de post-procesare. Explicați necesitatea utilizării CRF-urilor complet conectate în DeepLab, având în vedere că hărțile de scor CNN sunt **netede (*smooth*)**. Ce tip de informații utilizează *kernel*-ul bilateral al CRF pentru a rafina granițele?,,,

Răspuns II.3:

Deși CNN-urile DeepLab sunt excelente la clasificare, hărțile lor de scor oferă doar o **poziție brută (*rough position*)** și răspunsuri netede (*smooth responses*), neputând delimita precis granițele obiectelor. CRFs-urile tradiționale cu rază scurtă pot fi dăunătoare în acest regim, deoarece ar netezi și mai mult ieșirea. CRF-urile **complet conectate** sunt utilizate pentru a **cupla capacitatea de recunoaștere a CNN-urilor cu acuratețea de localizare fină (*fine-grained*)**,.

Kernel-ul bilateral (primul *kernel* Gaussian din potențialul de pereche) utilizează două tipuri de informații pentru rafinare: **poziția pixelilor ($\mathbf{p}_i$) și culoarea RGB ($\mathbf{I}_i$)**. Acest *kernel* forțează pixelii cu culoare similară și poziție similară să aibă etichete similare, ceea ce este vital pentru **recuperarea detaliilor fine ale granițelor**.

Întrebarea II.4 (DeepLabv3 și ASPP Îmbunătățit):

DeepLabv3 (7.8) revizuiește modulul ASPP. Care este problema care apare la rate mari de dilatație în ASPP și cum rezolvă DeepLabv3 această problemă prin introducerea **feature-urilor la nivel de imagine**?,

Răspuns II.4:

S-a descoperit că, pe măsură ce rata de dilatație (*sampling rate*) devine mai mare, **numărul de ponderi de filtru valide** aplicate regiunii de *feature* valide devine mai mic (în loc de zerouri *padded*).

Pentru a depăși această problemă și a încorpora informații de **context global** în model, DeepLabv3 adoptă **feature-uri la nivel de imagine (*image-level features*)**. Aceasta implică

aplicarea **global average pooling** pe ultima hartă de *feature*-uri, alimentarea rezultatului către o convoluție 1×1 cu normalizare pe batch și apoi eșantionarea în sus biliniar (*bilinearly upsample*) la dimensiunea spațială dorită, înainte de a fi concatenat cu celelalte ramuri ASPP,.

III. Exerciții de Calcul Aplicat

Exercițiul C.1 (Transposed Convolution Dimensionality):

Se aplică o convoluție transpusă pe un tensor de intrare X de dimensiune 40×40 cu c canale. Dacă s-a folosit un *stride* $s = 2$ (pentru ieșire), *padding* $s/2 = 1$, și un kernel de $2s = 4 \times 4$ într-o rețea care a generat $= f(X)$, determinați dimensiunea spațială a tensorului de ieșire $g()$, unde g este o convoluție transpusă cu aceiași *hyperparametri* (cu excepția numărului de canale).,

Rezolvare C.1:

Conform sursei, dacă se alimentează X unui strat convoluțional f pentru a obține $= f(X)$, iar apoi se creează un strat convoluțional transpus g cu aceiași *hyperparametri* (exceptând numărul de canale de ieșire egal cu numărul de canale din X), atunci $g()$ va avea **aceeași formă spațială ca** X,.

Prin urmare, dimensiunea spațială a tensorului de ieșire $g()$ va fi $\mathbf{40 \times 40}$.

(Alternativ, regula generală afirmă că pentru _stride s, *padding* $s/2$ și kernel $2s$, convoluția transpusă va crește înălțimea și lățimea intrării cu un factor de s. Dacă a fost redus cu un factor de $s = 2$, atunci ar fi 20×20. Aplicând convoluția transpusă cu $s = 2$ la rezultă $20 \times 2 = 40$. Ieșirea este 40×40)_.

Exercițiul C.2 (Dilated Convolution și Câmp Receptiv):

Se utilizează o convoluție dilatată (atrous) cu un kernel 3×3 ($k = 3$).

1. Determinați rata de dilatație r care trebuie aplicată pentru a obține o dimensiune efectivă a kernel-ului (k_e) de 15×15.
2. Explicați de ce această creștere a câmpului receptiv nu crește numărul de parametri învățabili ai filtrului.,

Rezolvare C.2:

1. **Calculul Ratei de Dilatație (r):** Formula pentru dimensiunea efectivă a kernel-ului este $k_e = k + (k-1)(r-1)$.

$$15 = 3 + (3-1)(r-1)$$

$$15 = 3 + 2(r-1)$$

$$12 = 2(r-1)$$

$$6 = r - 1$$

$$r = \mathbf{7}$$

2. **Constanța Parametrilor:** Deși dimensiunea efectivă a filtrului crește (la 15×15), convoluția dilatată introduce $r-1$ zerouri între valorile consecutive ale filtrului. Prin urmare, se iau în considerare doar valorile de filtru **non-zero** (adică cele 9 ponderi originale ale filtrului 3×3). Acest lucru menține **atât numărul de parametri de filtru, cât și numărul de operații per poziție constant**,.

Exercițiul C.3 (SegNet Max-unpooling):

O rețea SegNet aplică *max-pooling* 2×2 cu *stride* 2 pe o hartă de *feature*-uri 4×4. Encoder-ul stochează indicii de *max-pooling*.

Dacă o regiune 2×2 din harta de *feature*-uri de ieșire a Decoder-ului ar trebui să conțină valoarea 5, și indicele memorat indică poziția de jos-dreapta (colț) din regiunea 2×2 corespunzătoare, reprezentați matricial harta de *feature*-uri 4×4 rezultată după *Max-unpooling* (ignorând celelalte regiuni).,

Rezolvare C.3:

Max-unpooling plasează valoarea (5) doar în poziția memorată; celelalte poziții sunt zero,.

Dacă ne concentrăm doar pe regiunea 2×2 din dreapta jos a ieșirii 4×4:

$$\begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & \mathbf{5} \end{pmatrix}$$

Exercițiul C.4 (FCN Channels și Loss Function):

1. Dacă se antrenează un FCN (Fully Convolutional Network) pe un set de date care conține 15 clase de obiecte de interes, câte canale de ieșire (output channels) trebuie să aibă stratul final pentru predicția clasei?
2. Care este funcția de pierdere (loss function) utilizată la antrenarea FCN, și cum este calculată acuratețea?,

Rezolvare C.4:

1. **Numărul de Canale de Ieșire:** Numărul total de canale este dat de numărul de clase de obiecte plus clasa de *background*.

$$\text{Canale Ieșire} = 15 \text{ (clase obiecte)} + 1 \text{ (bacground)} = \mathbf{16}$$

(Similar cu Pascal VOC2012 care folosește 20 de clase + *background* = 21 canale).

2. **Funcția de Pierdere și Acuratețea:** Funcția de pierdere este **entropia încrucișată (*cross-entropy*) per pixel**. Acuratețea este calculată pe baza corectitudinii clasei prezise pentru **toți pixelii**,.